

University of Glasgow | School of Computing Science

Honours Individual Project Dissertation

A Comparative Evaluation of Smart, Traditional, and Software-Based Energy Meters

Satveer Sahaas Ganasan

September 14, 2018

Abstract

As energy consumption continues to rise globally, efficient monitoring and management of electricity usage have become crucial for businesses and consumers. Smart meters, traditional meters and, software-based energy meters have gained widespread adoption, offering advanced tracking capabilities and real-time insights into energy usage. However, these meters may sometimes produce inaccurate or unreliable results. Inaccurate readings may lead to misleading cost estimates and inefficient energy management strategies. This study conducts a comparative analysis of traditional, smart and software-based meters to evaluate their accuracy, usability, and effectiveness in measuring energy consumption. Our findings indicate that smart meters generally provide more precise and consistent readings than traditional meters. Additionally, unlike traditional meters, smart meters offer logging capabilities for storing and tracking data. Furthermore, software-based meters, such as HWiNFO and Open Hardware Monitor, demonstrate varying degrees of accuracy, with HWiNFO proving to be more reliable and consistent in its measurements. By identifying strengths and weaknesses in different types of meters, this research can provide valuable insights for researchers, consumers, and businesses seeking to enhance energy monitoring and make more informed energy management decisions.

Acknowledgments

I would like to thank Yehia Elkhatib for his support in aiding me in this project. Additionally, I would like to thank the participants who helped me with my usability survey.

Education Use Consent

I hereby grant my permission for this project to be stored, distributed and shown to other University of Glasgow students and staff for educational purposes. **Please note that you are under no obligation to sign this declaration, but doing so would help future students.**

Signature: Satveer Sahaas Ganasan Date: 28 March 2025

Contents

1 Introduction **1**
1.1 Motivation 1
1.2 Aim and Objectives 2

2 Background and related work **3**
2.1 What are Software, traditional and smart meters? 3
2.1.1 What are traditional meters? 3
2.1.2 What are Smart meters? 3
2.1.3 What are Software meters? 4
2.2 Related studies and potential gaps 4
2.2.1 An introduction to using a software meter such as Power API 5
2.2.2 Comparing different software meters 6
2.2.3 Evaluation of Intel RAPL for power measurement 6
2.3 Factors affecting accuracy testing and usability testing 7
2.3.1 Factors affecting accuracy testing 7
2.3.2 Factors affecting usability testing 8
2.4 Summary 9

3 Research methodology **10**
3.1 Research design 10
3.2 Experiment setup 11
3.2.1 Smart meter vs traditional meter experiment setup 11
3.2.2 Software meter vs software meter for accuracy testing 12
3.2.3 Software meter vs software meter for usability testing 13

4 Analysis of results **15**
4.1 Research 1: Smart meter versus traditional meter 15
4.1.1 Kettle test results 15
4.1.2 Rice cooker test results 16
4.1.3 Smart watch traditional meter vs smart meter 16
4.1.4 Laptop idle state traditional meter vs smart meter 17
4.1.5 Laptop CPU stressor state traditional meter vs smart meter 18
4.1.6 Summary of tests traditional meter vs smart meter 20
4.2 Research 2: Software meter versus software meter in Intel CPU 21
4.2.1 Laptop idle state results 21
4.2.2 Laptop gaming state results 22
4.2.3 Laptop CPU stressor with 5 processes and Idle threading results 23
4.2.4 Laptop CPU stressor with 10 processes and low threading results 23

4.2.5 Laptop CPU stressor with 15 processes and normal threading results 24
4.2.6 RMSE analysis for CPU tests 25
4.2.7 Laptop GPU game state test results 26
4.2.8 Laptop GPU heavy load stressor test results 26
4.2.9 Laptop GPU Furmark stressor test results 27
4.2.10 RMSE analysis for GPU tests 28
4.3 Research 3: Software meter versus software meter in AMD CPU 28
4.3.1 Laptop idle state results 28
4.3.2 Laptop game state results 29
4.3.3 Laptop CPU stressor with 5 processes and idle threading results 30
4.3.4 Laptop CPU stressor with 10 processes and low threading results 30
4.3.5 Laptop CPU stressor with 15 processes and normal threading results 31
4.3.6 RMSE of CPU results 32
4.3.7 Laptop GPU game test results 33
4.3.8 Laptop GPU Furmark stress test results 33
4.3.9 RMSE of GPU results 34
4.4 Research 4: Software meter versus Software meter usability testing results 35
4.4.1 Quantitative analysis 35
4.4.2 Qualitative analysis 36
4.4.3 Summary 37

5 Conclusion 38
5.1 Summary 38
5.2 Reflection 39
5.3 Future work 39

Appendices 41

A Graphs related to traditional vs smart meters 41

B Graphs related to software vs software meters in Intel CPU 44

C Graphs related to software vs software meters in AMD CPU 46

D Google Survey questions screenshot 48

E User manual to run the code 59

Bibliography 60

1 Introduction

In the modern era, smart meters, traditional meters, and software-based energy meters are widely adopted for various purposes, including gaming, energy management, and monitoring electricity usage. These meters enable users to track energy consumption, optimize efficiency, and even detect potential faults. However, such meters might produce faulty readings, resulting in ineffective energy management decisions.

This project aims to identify inconsistencies and unreliable readings between smart meters and traditional meters. Additionally, we will also have a comparison between software meters in terms of accuracy, consistency and usability. Through data analysis and visualization, we will explore these differences in depth to provide meaningful insights.

This chapter outlines the motivation for assessing the accuracy and usability of different meter types. Furthermore, we present the key objectives that this study aims to address.

1.1 Motivation

In the modern era, global energy consumption is increasing at an unprecedented rate, driven by the rapid expansion of data centers, solar panel production, electric vehicles (EVs), artificial intelligence (AI), and cloud computing. According to a report by International Energy Agency (IEA) (2025), China, in particular, has experienced a significant rise in electricity consumption since 2020, with a notable 7% increase in 2024 alone. This trend is not limited to China; the United States of America(USA) has also seen a substantial rise in energy demand over recent years, largely due to the increasing adoption of high-energy-consuming technologies.

A major factor behind this surge is the expanding footprint of data centers, which require vast amounts of electricity to power high-performance computing and cooling systems. According to a report by Poudineh (2025), data centers currently account for a significant share of global energy consumption, and this could rise to 10% of total electricity demand by 2030 as their numbers continue to grow. According to PricewaterhouseCoopers (PwC) (2025), the proliferation of AI and cloud-based technologies has further amplified energy consumption, as businesses increasingly rely on power-intensive machine learning model training, large-scale website deployments, and cloud computing services. These computationally demanding processes necessitate a robust and efficient energy infrastructure to support their operation. The rising energy consumption is not limited to businesses alone; individual consumers, including users of laptops, mobile phones, and household appliances, have also contributed significantly to the increasing demand for electricity. The household sector alone accounts for approximately 30% of global energy consumption, as reported by Wang et al. (2024).

With increasing energy consumption, this can lead to a constraint in natural resources which forces electric companies to increase the cost of electricity (Raza et al. (2023)). Thus forcing companies and even consumers to actively seek for ways to optimize energy usage and reduce unnecessary consumption. As a result, businesses are increasingly turning to smart meters, software-based energy tracking solutions, and traditional metering systems to monitor and manage their electricity usage more efficiently. In 2023, about 34.8 million smart meters were

installed in households and small businesses in just Britain alone (UK Government (2023)). Accurate energy readings have become essential for effective energy management, particularly for large enterprises where energy efficiency directly impacts operational costs and sustainability.

However, accuracy readings in various smart, traditional, and software meters may differ. Certain meter models might potentially provide much more consistent and accurate than the other. According to IoT For All (2025), seven out of nine smart meters were found to display false or inaccurate readings, raising concerns about their reliability. Such inaccuracies can have serious financial implications for both consumers and businesses, leading to inflated electricity bills, inefficient energy management strategies, and potential regulatory challenges.

Additionally, usability is a crucial factor in the development of any product, including software-based energy meters, as user interaction plays a key role in system effectiveness (Olivares Rojas et al. (2020)). If a software meter is not intuitive or lacks a user-friendly interface, it may discourage adoption, leading consumers to seek alternative solutions that offer better accessibility and ease of interpretation.

Therefore, ensuring the accuracy and usability of metering systems is therefore crucial for maintaining cost efficiency, encouraging widespread adoption, and fostering trust in modern energy monitoring solutions.

1.2 Aim and Objectives

The aim of this research is to evaluate the reliability, accuracy, and usability of different energy metering technologies such as traditional, smart, and software-based meters, to inform future developments and guide effective selection of tools for energy monitoring across modern computing environments.

To achieve this aim, the research will pursue the following objectives:

- Assess and compare the measurement accuracy and consistency of traditional and smart energy meters across various usage conditions.
- Evaluate the performance of software-based energy meters in terms of accuracy and consistency on different CPU architectures.
- Investigate how software-based meters vary in terms of usability, including user interface design and ease of interpretation.
- Identify key discrepancies and limitations across meter types that may impact their effectiveness for consumers and developers.

Understanding the accuracy and usability of different energy meters is crucial in an era where energy consumption is rapidly increasing, and efficient energy management is essential for both consumers and businesses. Smart meters are widely promoted as an advanced solution, but questions remain regarding their reliability compared to traditional metering systems. Additionally, as software-based energy tracking solutions gain popularity, their accuracy, consistency, and user experience must be assessed to determine their effectiveness.

Our findings will contribute to ongoing research efforts by providing a comparative analysis of the accuracy and consistency of different smart meter brands, as well as assessing the usability of software-based energy meters. By identifying potential discrepancies in accuracy across various metering solutions and evaluating their ease of use, this study aims to support both consumers and businesses in selecting the most reliable and user-friendly energy tracking technologies. Furthermore, the results may offer valuable insights for manufacturers and researchers working on improving metering systems for more precise and accessible energy management.

2 Background and related work

This chapter provides an overview of smart, software-based, and traditional energy meters, highlighting their functionalities and applications. It then examines key issues related to accuracy and usability in energy metering systems, addressing their impact on consumers and businesses. Furthermore, we review existing research in this field, identifying gaps that require further investigation. Finally, we discuss the key factors influencing accuracy and usability testing, along with the challenges involved in conducting comparative experiments on energy meters.

2.1 What are Software, traditional and smart meters?

In this section we will provide a small introduction towards software, traditional and smart meters.

2.1.1 What are traditional meters?

A traditional meter, also known as an analog or electromechanical meter, is used to measure electricity or gas consumption in homes and industries (Sharman (2025)). These devices use mechanical parts(specifically a spinning aluminum disc driven by magnetic fields from voltage and current coils) to calculate power consumption based on the formula P=V×I, where V is the voltage and I is the current. The speed of the disc's rotation correlates with the amount of energy used, which is displayed on a numerical register in kilowatt-hours (kWh) (Baah and Nwagbo (2022)).

While traditional meters are simple and reliable, they lack features like data storage, remote monitoring, or detailed consumption tracking. They only display real-time energy usage on a screen and require manual reading. These limitations have led to the rise of smart and software-based meters, though traditional meters laid the groundwork for modern metering technologies.

As shown in Figure 2.1, the device is plugged into a wall socket, and an appliance (e.g., a kettle) is connected to it. The screen displays live energy consumption as long as the appliance is running.

2.1.2 What are Smart meters?

A smart meter is a digital device that records electricity, gas, or water consumption in real time and transmits this data wirelessly to utility providers. Building on the foundation of traditional meters, smart meters address limitations such as the lack of automated readings and data storage. They enable remote monitoring, more accurate billing, and integration into smart grid systems for improved energy efficiency.

Smart meters, however, have their own limitations. Since they rely on Wi-Fi or cellular networks for communication, they are vulnerable to connectivity issues and potential cybersecurity threats, such as data interception (Khattak et al. (2019)). Additionally, smart meters have a shorter lifespan(typically 5 to 7 years) as opposed to traditional meters, which can last up to 40 years (Bimenyimana and Asemota (2018)). This implies that companies will need to take extra precautions and potentially invest more when using smart meters.



Figure 2.1: *Example of traditional meter*

While the core measurement mechanism remains similar to traditional meters, smart meters digitize, process, and transmit data automatically, removing the need for manual readings. As shown in Figure 2.2, smart meters usually lack a physical display and instead require users to access real-time data via companion apps or energy management systems. These apps communicate with the meter using various wireless protocols depending on the device's design.

2.1.3 What are Software meters?

A software meter is a digital tool that monitors and records energy consumption directly from a computer or electronic device. Unlike traditional and smart meters, which measure electricity usage of different electrical applications, software meters analyze power consumption at the system level, typically by tracking CPU, GPU, and component power draw.

Software meters like HWiNFO, Open Hardware Monitor(OHM) or even Intel Power Gadget use built in sensors within the computer to estimate power consumption in real time. For Intel CPU's, these software meters take data from Intel Running Average Power Limit(RAPL). Intel RAPL is a tool or sensor to take real-time measurements of CPU and RAM. These software meters provide logging capabilities to track the power consumption of various sensors over time, which allows users to use software tools to visualize CSV data.

Software meters are restricted to the device they are monitoring, which in this case is a PC or laptop. Additionally, some software meters are limited to tracking power consumption for specific CPU types, such as Intel processors. Different motherboard, CPU, and GPU manufacturers provide varying levels of sensor accuracy and compatibility. However, despite such limitations, many consumers use such meters for gaming, data centers, performance benchmarking, and more.

2.2 Related studies and potential gaps

This section presents a review of related studies on smart meters and software-based energy meters, analyzing their findings and limitations. By evaluating previous research papers, this study aims to identify key gaps in accuracy, usability, and standardization that remain unresolved. These gaps will serve as the basis for this research and guide the investigation in improving the



(a) *Example of smart meter side view* ***(b)*** *Example of smart meter front view*

Figure 2.2: *Example of smart meter*

reliability of the metering and the user experience.

2.2.1 An introduction to using a software meter such as Power API

Fieni et al. (2024) introduced PowerAPI, a software-based tool designed to overcome a key limitation of traditional and smart meters, specifically, their inability to measure power consumption at the process or component level. Their work argues that while traditional and smart meters can capture system energy usage, they lack the granularity required to attribute energy costs to specific applications or software services. PowerAPI addresses this by using Hardware Performance Counters (HwPC) and tools such as Intel RAPL, Perf, and Cgroup to collect raw system metrics, which are then processed using machine learning models to estimate energy usage at the CPU or GPU level. The study is valuable in two ways. First, it demonstrates the growing importance of software-defined meters in providing fine-grained energy insights. Second, it presents a hybrid hardware-software methodology that uses real-time monitoring of web applications to validate its output. However, several key limitations reduce the applicability of the findings.

Firstly, PowerAPI is built using hardware-specific tools and models (e.g., Intel RAPL), which limits its portability across different processor families or architectures. Fieni et al. (2024) themselves note that their models are statically trained and become unreliable under varying system conditions (e.g., memory load or application type), highlighting an important challenge in software metering. Secondly, while the study effectively demonstrates PowerAPI's capabilities, it lacks any comparative evaluation. It does not assess how PowerAPI performs against other popular software meters such as HWiNFO or Open Hardware Monitor. As a result, the reader is left without a clear understanding of PowerAPI's relative accuracy or consistency.

This creates a clear research gap, while Fieni et al. (2024) provide a compelling case for software-based metering, they do not explore how tools differ in performance, nor how such tools behave across different CPU architectures. This is precisely where our study contributes. By comparing

multiple software meters (HWiNFO vs OHM) across both Intel and AMD platforms, we evaluate the consistency, accuracy, and usability of these meters under real-world and bench-marked scenarios.

In summary, Fieni et al. (2024) highlight the need for fine-grained software energy monitoring and propose a technically sound solution. However, their approach is narrow in scope and lacks cross-tool comparisons, platform diversity, and user-facing evaluation. These limitations directly shaped the objectives of our study, which focus on evaluating software-based power meters across multiple platforms, tools, and use cases.. Furthermore, their methodologies provide a baseline for the design of our own methodological approach.

2.2.2 Comparing different software meters

A study by Jay et al. (2023) offers a comprehensive evaluation of various software-based power metering tools, making it highly relevant to the objectives of this research. The authors compare a broad range of power monitoring tools(such as Green Algorithms, Carbon Tracker, and PowerAPI) focusing on their accuracy, estimation models, usability, and hardware compatibility. Their aim is to support developers and data center managers in selecting suitable metering solutions by categorizing meters based on their modeling approaches (e.g., TDP-based models vs hardware counters like Intel RAPL or NVIDIA NVML) and evaluating their performance under different workloads.

The paper's key strength lies in its structured benchmarking methodology, where each software meter is tested under varied CPU and GPU workloads. This revealed notable variations in power estimation accuracy, with some tools overestimating or underestimating depending on system configuration and application type. The authors also emphasize usability aspects, such as the complexity of setup and configuration, offering practical insights that are often overlooked in technical evaluations. Moreover, their use of visualizations and workload diversity influenced the methodology of our research, particularly in terms of system level testing(such as CPU and GPU test) and how to present energy data effectively.

However, the study has notable limitations. Most significantly, it does not explore how software meters behave across different hardware platforms. While several tools are bench-marked, they are not tested across CPU architectures (e.g., Intel vs AMD) or varying GPU environments. This creates a gap in understanding whether certain meters are more or less reliable depending on the underlying hardware. Furthermore, while the study is thorough in evaluating multiple tools, it does not conduct ground comparisons using hardware-level monitoring (e.g., Intel RAPL or uProf), making it difficult to judge the absolute accuracy of each tool's estimation. Our research builds upon and extends the work of Jay et al. (2023) by directly addressing these gaps and utilizing the research methodology provided by the authors.

In summary, the study by Jay et al. (2023) offers a solid foundation for understanding the current landscape of software-based energy metering. However, its lack of architecture-specific comparisons and hardware-grounded benchmarks leaves important questions unanswered, these are questions that our research directly aims to resolve.

2.2.3 Evaluation of Intel RAPL for power measurement

Another highly relevant paper to my research was done by Khan et al. (2018), which investigates the accuracy, reliability, and limitations of intel RAPL interface for energy measurement. RAPL is widely used in power monitoring and modeling due to its ability to provide real time power consumption data for different CPU components, including DRAM, processor cores and more. However, whilst RAPL is frequently utilized in research and industry, its weakness and strengths has not been comprehensively analyzed. Therefore, to fill such gap the authors conduct experiments to evaluate intel RAPL's effectiveness in different computing environments.

The experiments were done utilizing custom made micro-benchmarks as well as three well known application level benchmarks to assess how accurately RAPL measures power consumptions under different workloads. The author will try and test RAPL across different hardware architectures, including Intel Haswell, Intel skylake and Intel Sandy Bridge. This inspired me to try and do various test of software meters in different CPU types which in this case I chose AMD Ryzen and Intel CPU. The experiments are between Intel RAPL and high precision external power meters when comparing the accuracy of RAPL. This will further fortify the usage of Intel RAPL in our experiment as a baseline when comparing the data.

Overall, this paper provides a thorough examination of RAPL's capabilities, highlighting both its advantages and its limitations. The study's findings are highly relevant to my research, as RAPL serves as a key reference point for evaluating software-based power meters. While the study confirms RAPL's accuracy and efficiency, it also identifies potential weaknesses, reinforcing the need for further validation when using RAPL as a ground-truth energy measurement tool. Our research builds upon this by comparing software meters such as HWiNFO and Open Hardware Monitor against RAPL, assessing how closely their energy readings align with RAPL's estimations.

2.3 Factors affecting accuracy testing and usability testing

This section will describe the factors that will affect the readings of accuracy and usability testing. We will first introduce factors which will affect accuracy testing in traditional, smart and software meters. Finally we will explain factors which will affect usability testing.

2.3.1 Factors affecting accuracy testing

When comparing the accuracy between different meters there are plenty of things which can affect the experiments, in the case of software meters this can be temperature of laptop, hardware types and more. The first main factor to consider the software meters to use when testing on different CPU's. The reason to this is because most software meters do not support certain CPU's. Therefore, using a software meter which can be supported in the relevant CPU chosen(In our case AMD and Intel) is crucial. Additionally, the chosen software meters should measure a variety of things, not just CPU consumption, but GPU consumption as well. Reason to such is due to the fact we want to measure both CPU and GPU and see how consistent the measurements of the software meters are across different system components.

Another critical factor affecting accuracy is the operating temperature of the system during testing. As identified by Khan et al. (2018), energy consumption readings can be influenced by elevated temperatures. This is because high temperatures may trigger thermal throttling mechanisms, causing the CPU to limit its power consumption, which in turn affects the readings recorded by software meters. Additionally, some software meters may struggle to measure energy consumption accurately under high-temperature conditions due to sensor inaccuracies or delayed updates. Therefore, ensuring a stable and controlled temperature environment is essential when conducting accuracy tests to prevent external thermal factors from skewing results.

A further crucial aspect of accuracy testing is sampling frequency, which refers to how often a meter updates power consumption data. As discussed in Khan et al. (2018), Intel RAPL offers a higher sampling rate compared to external power meters, allowing it to capture rapid power fluctuations with greater precision. Some software meters update power readings every few milliseconds, while others aggregate energy usage over longer intervals. Meters with lower sampling rates may fail to capture short bursts of high power consumption, leading to underestimations of peak energy usage. To ensure fair comparisons, it is important to select software meters with similar sampling intervals, preferably meters that log data at the same second-interval rates.

Although many of the factors affecting software meters also apply to smart meters and traditional meters, these hardware-based metering systems come with their own unique challenges in accuracy testing. One of the primary challenges in traditional meters is their lack of data transmission and storage capabilities, meaning that readings must be manually recorded. Unlike smart meters, which can transmit data digitally, traditional meters only display real-time energy consumption, requiring video recording or frequent manual observation to track usage over time. This introduces the risk of human error, particularly if readings are not recorded at precise intervals, leading to inconsistencies in accuracy assessments.

Another key factor affecting accuracy in both smart and traditional meters is calibration and response time. Traditional meters often rely on mechanical components, such as rotating disks or dials, to measure energy consumption, which may introduce delays in reflecting rapid power fluctuations. On the other hand, smart meters provide real-time digital readings, but their accuracy may be affected by firmware inconsistencies, network latency, or data transmission errors. If a smart meter lags in reporting power usage due to a slow transmission interval, it may fail to capture short-term spikes in energy consumption, leading to underestimations of peak demand.

2.3.2 Factors affecting usability testing

Usability testing for software-based power meters is crucial to ensuring that these tools are accessible, efficient, and user-friendly for a wide range of users, including researchers, data analysts, and system administrators. Unlike traditional and smart meters, which primarily focus on hardware-based measurements, software meters rely on graphical user interfaces (GUIs), command-line interfaces (CLIs), and data visualization tools. Therefore, usability testing must assess factors such as ease of installation, interface design, learning curve, compatibility, and data interpretation.

In addition to evaluating installation, interface design, data visualization, and performance, usability testing must also consider methodological factors, such as sample size, participant selection, and testing approaches. These factors are essential in ensuring that the usability study provides meaningful, representative, and actionable results.

A paper by Fox (2015) stated factors to identifying the right sample size. Furthermore, Fox (2015) stated that not only are the factors to identifying the sample size matter, a huge consideration to note will be time constraints and cost. Therefore, as suggested by Fox (2015) a recommended guideline for the experiment is to get about 20 participants when doing quantitative tests.

Beyond sample size, participant selection plays a critical role in usability testing outcomes. As highlighted by Fox (2015), the experience level of participants can significantly affect usability results, as trained users are more likely to navigate interfaces efficiently, potentially overlooking usability issues that novice users may encounter. Therefore, recruiting participants who lack prior exposure to software meters can help reduce bias and provide a more accurate assessment of usability challenges.

Further supporting this, research by Nielsen (2025) emphasizes the importance of user diversity, suggesting that usability testing should include participants with varying levels of technical expertise to capture a wider range of usability problems. By incorporating both untrained and trained users, we can better understand how different user demographics interact with the software, ensuring that the findings are applicable to real-world usage scenarios.

Overall, selecting the sample size and type of sample is a crucial factor to consider when doing usability testing. Furthermore, considerations to time must be taken note of when selecting a relevant sample size as well as diversity of a sample. Choosing large samples may not be realistic for a short time constraint and the same can be said for a extremely diverse sample.

2.4 Summary

In summary, this chapter has critically reviewed existing literature on traditional, smart, and software-based energy meters, highlighting their respective capabilities and limitations. Several studies have laid the groundwork for understanding power measurement tools, but notable gaps remain, particularly in comparing meters across different hardware platforms and evaluating both accuracy and usability. These gaps provide the foundation and motivation for our research. As we move forward, the reader should keep in mind the central role of consistency, accuracy, hardware sensitivity, and user experience in assessing the effectiveness of smart meters.

3 Research methodology

As highlighted earlier, our goal for this research is to identify which meters are more accurate and usable. We provided a list of points we wished to identify and we aim to fill in the gaps of previous research or even provide new insights to consumers, businesses, or even researchers.

This chapter provides a detailed overview of the research methodology employed to achieve these objectives. First, we outline the overall research design and explain the rationale for our chosen approach. Then, we describe the experimental procedures for both accuracy testing and usability testing, detailing the specific metrics and conditions under which these evaluations were conducted. Additionally, we elaborate on the data collection techniques used to ensure consistency and reliability in measurement. Finally, we discuss the data analysis methods and evaluation metrics applied to interpret the results.

3.1 Research design

This study employs a quantitative experimental research design to evaluate and compare the accuracy and usability of smart meters, traditional meters, and software-based meters. A quantitative research design is most appropriate for this study because it allows for objective numerical comparisons of energy meter accuracy and usability. Additionally, a comparative methodology was chosen to systematically measure and analyze energy readings across different metering technologies. Finally, usability testing was conducted to assess ease of use, accessibility and overall user experience of software meters.

The research is divided into two main components which we will discuss more about in the following sections.

- Accuracy testing - a series of controlled experiments were conducted to measure and compare power consumption readings of different meters under various workloads.
- Usability testing - participants were asked to interact with selected software meters to assess usability factors such as installation complexity, ease of navigation and more.

The study mainly follows a comparative analysis framework, where multiple energy meters were tested under identical conditions to ensure fair and consistent evaluation. The experiments were conducted using a structured experimental protocol which we will elaborate more about in the following sections. We repeated the tests plenty of times to minimize variability in results as well as ensure a fair test is done. The data collected from these experiments where analyzed using statistical methods and visualization techniques.

There are various tests done on each different meters and I will highlight the tests done below, an explanation of each test will be explained in the following sections.

- Smart meter vs traditional meter
- Software meter vs Software meter in Intel CPU
- Software meter vs Software meter in AMD CPU
- Software meter vs Software meter usability testing

3.2 Experiment setup

In this section, we will further elaborate on the experiment setup of the proposed experiments stated above. We will discuss data collection methods at each section, hardware used, and more.

3.2.1 Smart meter vs traditional meter experiment setup

Experiment setup and rationale The goal of this experiment is to evaluate and compare the accuracy and consistency of a traditional energy meter (Maxcio PM 01) and a smart meter (Kasa KP-115) under controlled load conditions. This directly supports our objective of determining whether traditional meters remain more reliable than newer smart technologies. Due to a lack of formal academic comparisons between these meter types, our methodology was informed by industry recommendations and real-world testing practices.

As outlined by Good Energy (2025) and LinkedIn (2025), benchmarking accuracy can be achieved by comparing meter readings to known manufacturer-specified energy consumption values. For instance, Good Energy (2025) suggests using a kettle, whose energy draw is generally consistent and documented, as a reliable test load. By comparing measured data from both meters against these known values, we can quantify their accuracy and detect inconsistencies.

Load conditions and duration To fulfill our objective of assessing meter accuracy and consistency under real-world usage scenarios, we selected a variety of energy loads that reflect both low and high consumption devices. As highlighted by Power Line (2021), testing under different load conditions is standard practice when evaluating energy meter performance. This approach allows us to examine whether certain meter types are more accurate or stable under specific power draws.

Each experiment was designed to last either 1 or 5 minutes depending on the appliance type. This duration was chosen to allow for stable readings without overextending the experiment time. All tests were repeated three times to ensure statistical reliability. The full list of load types and durations is shown below:

- Smart watch testing(low load) - 1 minute
- Laptop idle state testing(low load) - 5 minutes
- Rice cooker testing(medium load) - 1 minute
- Laptop running a game state testing(medium load) - 5 minutes
- Laptop maximum CPU stressor testing(medium load) - 5 minutes
- Toaster testing(high load) - 1 minutes
- Kettle testing(extremely high load) - 1 minutes

This setup enables us to examine how each meter handles different levels of energy demand, which is essential in addressing our research question on which type of meter is more consistent and accurate.

Data collection techniques To extract data from the meters, different approaches were required due to their distinct data retrieval mechanisms. For the smart meter (KP-115), a virtual environment was set up to host a local server, allowing access via a web service to retrieve energy consumption logs. This enabled data extraction in CSV format by specifying a start and end time, streamlining the process. However, traditional meters lacked digital connectivity, requiring a manual extraction method. Since manually transcribing readings from video recordings was impractical, an automated solution was developed, where frames were extracted every second and processed using a Vision-based Large Language Model (LLM) to detect and extract numerical readings. The extracted data was then structured into a CSV format, ensuring consistency and comparability with smart meter readings while minimizing human error.

Evaluation metrics We utilize a range of evaluation metrics to assess meter performance, aligning directly with our objective of identifying consistency and accuracy across metering solutions. Time series graphs are plotted to visually inspect fluctuations and detect any abrupt spikes in energy consumption over time. This allows us to view visually if the smart meter is consistent. Gaussian distribution curves and cumulative distribution functions (CDFs) are used to explore the spread and distribution patterns of readings, helping us understand whether measurements are tightly clustered or widely dispersed. This helps evaluate whether a smart meter consistently produces stable readings under varied conditions, which is a key component of our objectives of measuring accuracy and consistency within smart meters and traditional meters.

3.2.2 Software meter vs software meter for accuracy testing

Experiment setup and rationale As discussed in Section 2.2, previous research has established the importance of stress testing and load-based evaluation when analyzing the accuracy of software meters. Drawing inspiration from Fieni et al. (2024), who used server-based CPU stressors to benchmark software meter accuracy, we apply a similar experimental framework by introducing controlled CPU and GPU workloads. Additionally, following the approach of Jay et al. (2023), we ensure each software meter is tested under different load intensities to assess performance under variable conditions. This directly supports our objective of identifying which software-based meters maintain accuracy and consistency across diverse computational environments.

The meters selected for this study are HWiNFO and Open Hardware Monitor (OHM), which serve as the primary tools under evaluation. To assess their accuracy, we compare them against trusted baseline tools: Intel Power Gadget for Intel CPU's, Nvidia-SMI for GPU monitoring, and uProf for AMD CPU's. Intel Power Gadget leverages Intel RAPL, a widely accepted ground truth for CPU power measurements, as validated by Khan et al. (2018). Nvidia-SMI is an official NVIDIA telemetry tool that provides real-time GPU power data, making it a suitable benchmark for evaluating GPU-related energy readings. Meanwhile, AMD uProf serves as a low-level monitoring tool specific to AMD architectures, offering direct access to power telemetry and thus acting as our baseline for Ryzen-based systems. This multi-platform baseline strategy directly supports our objective of evaluating software meter consistency and accuracy across different hardware environments.

Load scenarios and test duration For the experimental setup, we will conduct tests on two laptops, one equipped with an Intel CPU and another with an AMD CPU, both featuring NVIDIA GPU's. We will perform multiple experiments on both CPU and GPU power consumption under varying workloads to ensure a comprehensive evaluation. Each test will run for five minutes, and each meter will be tested three times per workload condition to enhance reliability and ensure statistical consistency. The detailed breakdown of the test scenarios and measurement procedures will be presented below.

- idle test- for CPU and GPU
- game tests- for CPU and GPU
- CPU stressor 5 process idle threading - for CPU
- CPU stressor 10 process low threading - for CPU
- CPU stressor 15 process normal threading - for CPU
- CPU stressor memory testing 1000MB available memory and 250MB/s intensity - for CPU
- CPU stressor memory testing 1000MB available memory and 500MB/s intensity - for CPU
- CPU stressor memory testing 1000MB available memory and 750MB/s intensity - for CPU

- idle GPU stress test - for GPU
- game GPU stress test - for GPU
- GPU stress test - for GPU
- GPU stress using Furmark - for GPU

These scenarios were selected to reflect low, medium, and high energy consumption cases, which aligns with our goal of evaluating software meter performance under varied system loads.

Data collection techniques To extract data from the meters, the same approach was done where we will automatically log data provided by features from the relevant software meters. However, open hardware monitor did not give a automatic logging method therefore we developed a script to ensure such is done. Again all extracted data is stored in a CSV format, therefore ensuring consistency and comparability. This consistent data structure allows straightforward analysis and comparison across tools.

Evaluation metrics We selected a blend of visual and statistical metrics to evaluate meter accuracy and consistency. Root Mean Square Error (RMSE) is used to quantify the difference between HWiNFO and OHM readings versus baseline tools (Intel Power Gadget, Nvidia-SMI, uProf). RMSE provides a clear measure of deviation per second, enabling direct accuracy comparison. To assess consistency, we use standard deviation and visualize results through Gaussian distribution curves and Cumulative Distribution Functions (CDFs). These charts help identify measurement concentration and reveal how readings are distributed across experiments. Finally, RMSE values are summarized and plotted as bar charts for each system-level experiment (e.g., all CPU tests), giving a high-level view of performance differences between HWiNFO and OHM across workloads. This supports our goal of determining which software meter offers the most dependable performance across architectures and load levels.

3.2.3 Software meter vs software meter for usability testing

Experiment setup and reasoning The usability experiment setup is designed to evaluate how efficiently users can interact with HWiNFO and Open Hardware Monitor for energy monitoring. Unlike accuracy testing, which focuses on numerical precision, this experiment addresses usability. We assess the software meters based on four established usability categories: intention to use, perceived ease of use, perceived usefulness, and user satisfaction. These four usability categories originate from the Technology Acceptance Model (TAM), a widely recognized theory that explores the factors influencing user's acceptance and utilization of new technologies. Participants will perform a series of basic tasks, including installing the software, extracting power readings, and interpreting the interface output. Their performance and feedback help determine which tool offers the most intuitive and accessible experience, supporting our objective of assessing user-friendliness.

Experiment sampling We will recruit 20 participants from diverse academic backgrounds, ensuring a mix of technical (e.g., computer science, engineering) and non-technical (e.g., business, arts) disciplines. This diversity helps assess usability across varying levels of user experience. Participants unfamiliar with energy meters provide insights into learning curve and interface design, while more experienced users help validate advanced usability aspects. We recruited 10 participants from technical backgrounds and 10 participants from non-technical backgrounds, thus providing us a 50% split. This approach aligns with our goal to identify which software meter is suitable for a broad range of users which is part of usability as a good application can accommodate for a wide range of users.

Questionnaire design Participants will complete a structured survey hosted on Google Forms after using each tool. The questionnaire consists of 12 statements rated on a 5-point Likert scale, covering the four usability categories. This method enables scalable and anonymous feedback collection while minimizing researcher influence. The questions are:

- I intend to use this software regularly when monitoring system performance (Intention to use)
- Given the choice, I would recommend this software to others for hardware monitoring.(Intention to use)
- I thought the software was easy to use.(Perceived Ease of Use)
- I did need external help to set up the software (e.g. technical person, online help)(Perceived Ease of Use)
- Once the software is set up, I found it easy to use for my energy monitoring tasks.(Perceived Ease of Use)
- Most people would learn to use this software very quickly.(Perceived Ease of use)
- I found the software's interface to be user-friendly.(Perceived Ease of use)
- The software helped me accomplish my hardware monitoring tasks effectively. (Perceived usefulness)
- This software offered solutions that other similar tools do not provide. (Perceived usefulness)
- I found the information provided by the software to be accurate and reliable.(Perceived usefulness)
- I am happy with the software as it is. (User satisfaction)
- I would like the software to provide more monitoring information. (User satisfaction)

To enrich our understanding of user experience, participants will also perform a predefined set of basic tasks (e.g., installation, extracting readings, interpreting output) before answering the questions above and report any challenges they encounter. The tasks are shown in the table 3.1. These qualitative responses help identify usability obstacles and contextualize the survey ratings. This hands-on interaction ensures that their subsequent survey responses are grounded in actual usage, allowing us to identify specific usability obstacles and better contextualize the Likert scale ratings.

	Tasks
1	Install the given Software meter
2	Using the given software meter, identify your devices GPU and CPU
3	Using the given software meter, identify your CPU temperature usage
4	Using the given software meter, identify your CPU energy usage
5	Using the given software meter, identify your audio devices
6	Finally using the given software meter, try and extract or log data of your computer system for 5 seconds.

Table 3.1: *The tasks the users will do when they participate in the survey. In this case the given software meter is either HWiNFO or Open Hardware Monitor*

Evaluation Metrics For usability evaluation, responses to the Likert scale questionnaire will be aggregated and visualized using bar charts, offering a clear comparison of participant ratings across tools and categories. The average scores for each usability factor will be calculated per tool. To complement this quantitative data, we will perform thematic analysis on open-ended feedback to identify recurring usability issues such as unclear labels or difficult installation procedures. This dual approach ensures both measurable trends and deeper contextual understanding, supporting our objective of identifying the most user-friendly software meter.

4 Analysis of results

In this chapter we will go through the results of the various tests performed. Firstly we will discuss the smart meter versus traditional meter results. Next we will provide a comparison between software meters for energy consumption accuracy in different CPUs. Finally we will provide a comparison between software meters in terms of usability.

4.1 Research 1: Smart meter versus traditional meter

In this section, we will cover some of the tests results as mentioned in section 3.2.1.

4.1.1 Kettle test results

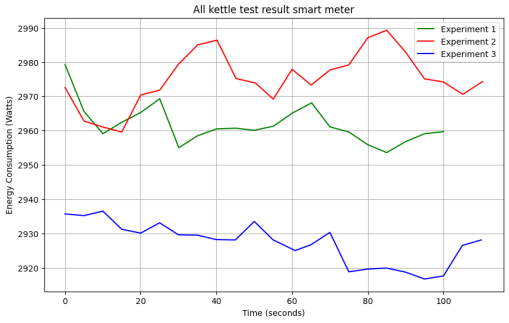


Figure 4.1: *Smart Meter kettle experiment time series*

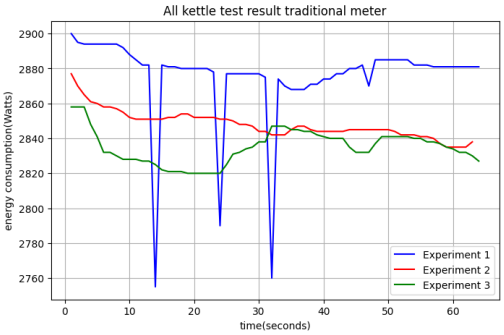


Figure 4.2: *Traditional Meter kettle experiment time series*

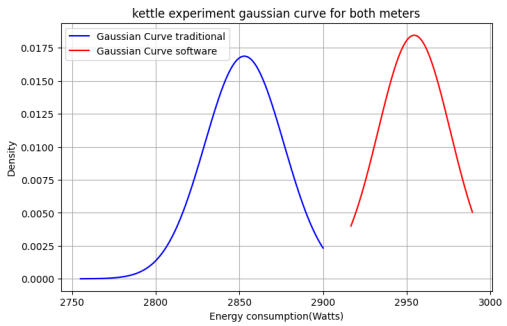


Figure 4.3: *Gaussian Distribution of both meters*

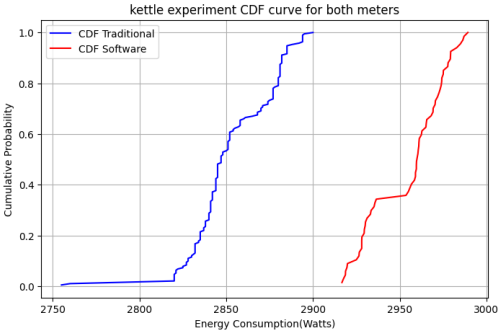


Figure 4.4: *CDF of both meters*

Figure 4.5: *Comparison of Smart and Traditional Meter Energy Measurements Across Different Analyses*

As shown in Figures 4.1 and 4.2, there are notable differences in the consistency and reliability of energy readings between the smart and traditional meters. The smart meter demonstrates

stable results across all experiments, with only minor deviations in Experiment 3, suggesting good internal consistency and responsiveness to the kettle's steady power draw. In contrast, the traditional meter shows substantial fluctuations, particularly in Experiment 1, where sharp drops were seen going from 2880W to 2760W suggest sampling resolution or response time issues. These inconsistencies undermine its reliability and may indicate difficulty capturing real-time fluctuations accurately. This directly contradicts expectations set by previous research from Durand et al. (2022), which confirms that kettles typically deliver consistent energy consumption. Therefore, this suggests traditional meter might be inconsistent when sampling results from high load experiments.

The distinction becomes clearer in the Gaussian distribution shown in Figure 4.3, where the curves for the two meters do not overlap. The smart meter has a narrow, concentrated peak, indicating low variance and high measurement repeatability. In contrast, the traditional meter's broader distribution implies a higher standard deviation and less consistent performance. This observation is reinforced by the CDF plot (Figure 4.4), where the smart meter's curve rises sharply between 2920W and 2980W, demonstrating a tight, reliable range of readings. Meanwhile, the traditional meter's CDF ascends more gradually over a wider range, from 2760W to 2890W, indicating broader variability. Additionally, we see that between around 2930W and 2960W in the smart meter, there is a slight incline line suggesting stable and consistent readings. While both meters fall within the expected manufacturer range of 2520W to 3000W, the smart meter's more stable output makes it the more reliable tool for accurate and repeatable measurements under high-load conditions like kettle usage.

4.1.2 Rice cooker test results

In Figure 4.6, the smart meter demonstrates consistent and smooth energy readings across all three experiments, fluctuating narrowly between 293W and 296W. This indicates high measurement repeatability and responsiveness to the rice cooker's steady power draw. Although Experiment 3 begins slightly lower, the variation is minor and likely influenced by startup conditions or external factors. In contrast, the traditional meter in Figure 4.7 displays abrupt, step-like changes rather than continuous trends, with readings ranging from 284W to 293W. This pattern suggests lower sampling resolution or slower response time, which undermines its ability to capture real-time variations accurately. These observations imply that under moderate energy loads(such as with the rice cooker) the traditional meter may have a reduced sampling rate, in contrast to its response under higher loads, such as with the kettle.

These observations are reinforced by the Gaussian and CDF plots (Figures 4.8 and 4.9). The smart meter's Gaussian curve is tightly clustered, indicating low variance and strong internal consistency, while the traditional meter's wider distribution reflects more variability in measurement. Similarly, the smart meter's CDF progresses smoothly, showing a stable and concentrated range of values, whereas the traditional meter's CDF rises in discrete jumps and has a step-like pattern, highlighting its lower granularity. Although both meters report values above the rice cooker's rated 250W, this may be due to minor inefficiencies, voltage supply or measurement tendencies. However, the smart meter's tighter distribution and finer sampling make it more reliable for accurate energy tracking under medium-load conditions.

4.1.3 Smart watch traditional meter vs smart meter

As shown in Figure 4.11, the smart meter consistently recorded smartwatch energy consumption between 1.4W and 1.8W across all three experiments. Although the time series curve is not entirely smooth, the observed fluctuations fall within a narrow, predictable range, indicating strong measurement stability. This suggests that the smart meter is sensitive enough to detect low-power devices like smartwatches and can produce repeatable measurements over multiple trials. In contrast, Figure 4.12 shows the traditional meter captured almost no usable data(registering

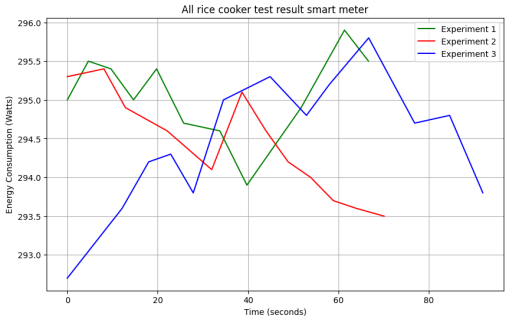


Figure 4.6: *Smart Meter rice cooker experiment time series*



Figure 4.7: *Traditional Meter rice cooker experiment time series*

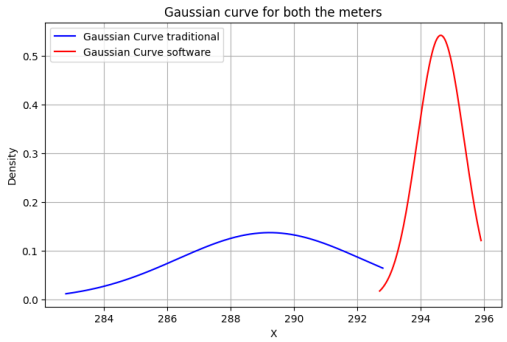


Figure 4.8: *Gaussian Distribution of both meters*

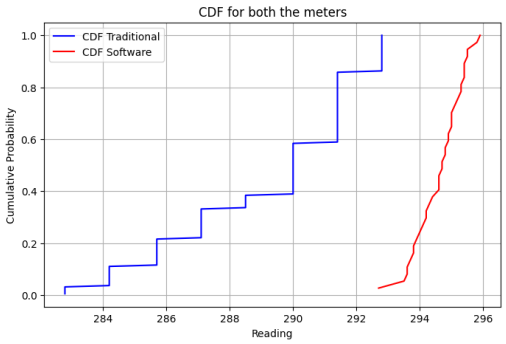


Figure 4.9: *CDF of both meters*

Figure 4.10: *Comparison of Smart and Traditional Meter Energy Measurements Across Different Analyses*

only a single spike at around 0.8W). This under-performance highlights the meter's inability to detect or respond to minimal loads, revealing a significant limitation in its applicability for modern low-energy devices.

This disparity is further illustrated in the distribution plots. Figure 4.13 shows a narrow, well-defined Gaussian curve for the smart meter, indicating consistent measurement density near the 1.5W–1.8W range. On the other hand, the traditional meter's Gaussian curve is heavily skewed toward 0W, reinforcing that it failed to detect the device in most samples. Similarly, Figure 4.14 demonstrates a smooth and steep rise for the smart meter's CDF, implying a tight distribution of values. Meanwhile, the traditional meter's CDF remains almost flat before an abrupt jump, signaling only a few registered values. This inability to capture low-power activity suggests traditional meters are not only outdated in design but fundamentally unsuitable for measuring increasingly common low-energy electronics in modern households.

4.1.4 Laptop idle state traditional meter vs smart meter

As shown in Figures 4.16 and 4.17, the energy readings during the laptop idle state differ significantly between the smart and traditional meters in terms of sensitivity and temporal resolution. The smart meter captures noticeable spikes in Experiment 1 and 3, with readings momentarily rising above 50W and even 70W, suggesting it is sensitive to transient system-level fluctuations that occur even during idle states. Conversely, the traditional meter shows a much flatter response, with readings consistently between 17W and 23W and only minor variations. While this may initially appear more stable, the step-like nature of the traditional meter's time



Figure 4.11: *Smart Meter smart watch experiment time series*

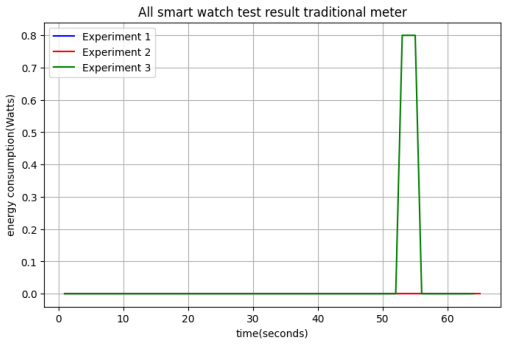


Figure 4.12: *Traditional Meter smart watch experiment time series*

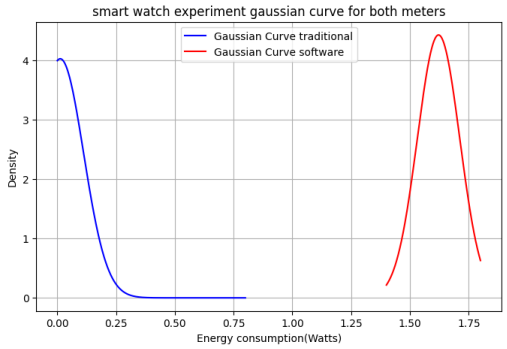


Figure 4.13: *Gaussian Distribution of both meters*

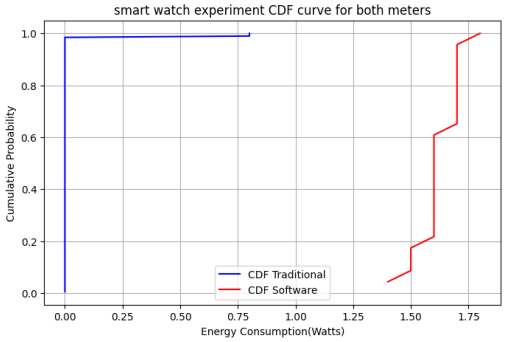


Figure 4.14: *CDF of both meters*

Figure 4.15: *Comparison of Smart and Traditional Meter Energy Measurements Across Different Analyses*

series curve indicates a coarse sampling rate, which may mask short duration changes in power usage and prevent it from capturing fine-grained fluctuations.

The difference in measurement sensitivity is more clearly illustrated in Figures 4.18 and 4.19. The Gaussian curve for the traditional meter is sharply peaked, reflecting tightly grouped values and a narrow standard deviation(signs of repeatability but potentially at the cost of missing real-time variability). In contrast, the smart meter's wider Gaussian curve indicates higher measurement variance, which might at first seem inconsistent but is better interpreted as responsiveness to fluctuations in energy consumption. This interpretation is reinforced by the CDF, where the traditional meter exhibits a steep rise with a subtle step-like progression, reflecting both uniformity and a lack of fine-grained sampling. Meanwhile, the smart meter's gradual and wider CDF slope suggests a broader spread of values. Given that laptops can experience variable background processes even at idle, the smart meter's distribution more likely reflects actual system behavior, whereas the traditional meter appears to under-represent these subtle fluctuations due to its limited resolution.

4.1.5 Laptop CPU stressor state traditional meter vs smart meter

As shown in Figure 4.21, the smart meter demonstrates noticeable fluctuations during the CPU stress test, especially in Experiment 1, where sharp peaks and drops indicate responsiveness to real-time changes in workload. While Experiments 2 and 3 exhibit slightly less variation, all three maintain readings within a tight 140W–160W band, reflecting consistent measurement under stress conditions. This behavior suggests the smart meter is sensitive enough to register

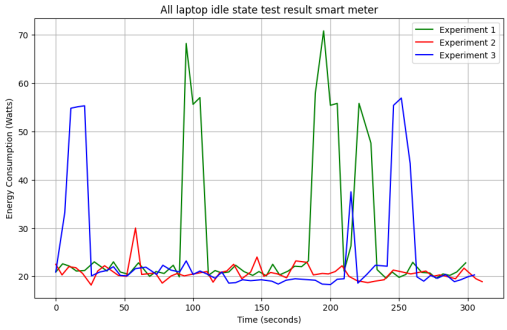


Figure 4.16: *Smart Meter laptop idle state experiment time series*

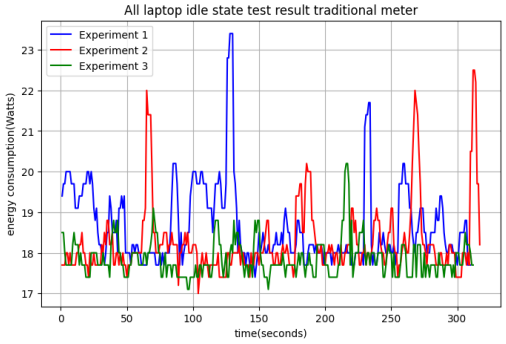


Figure 4.17: *Traditional Meter laptop idle state experiment time series*

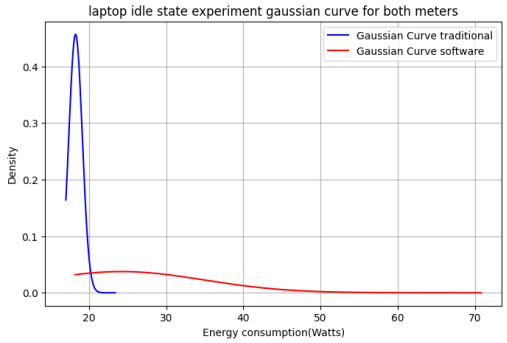


Figure 4.18: *Gaussian Distribution of both meters*

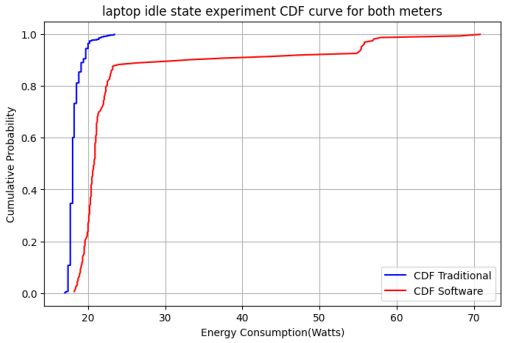


Figure 4.19: *CDF of both meters*

Figure 4.20: *Comparison of Smart and Traditional Meter Energy Measurements Across Different Analyses*

transient spikes that occur during processor load changes. In contrast, the traditional meter (Figure 4.22) presents a flatter profile with a step-like pattern, indicative of lower sampling frequency. Although its readings appear more stable just like in the previous experiment, this stability comes at the expense of missing finer variations. More critically, the traditional meter reports values both below 100W and above 160W, which introduces questions about its accuracy and consistency. These outlier values may reflect either delayed sampling or poor responsiveness to rapidly shifting CPU activity.

This inconsistency is further illustrated in the Gaussian distribution (Figure 4.23), where the smart meter forms a tight, symmetrical peak centered near 150W, confirming its high repeatability. Conversely, the traditional meter's curve is noticeably wider and skewed toward lower power readings, implying less reliable measurement and a tendency to under-report consumption. This is reinforced in the cumulative distribution function (Figure 4.24), where the smart meter's steep CDF slope reflects a dense cluster of values around the mean. The traditional meter's CDF, by contrast, rises gradually and in steps, further supporting the conclusion that it lacks resolution to track real-time variation. Collectively, these results suggest that while the traditional meter may provide a general sense of power draw, it is not suitable for high-resolution tracking in dynamic environments like CPU stress tests. Traditional meters seem to produce less sensitive results under medium to low energy load as opposed to the smart meter. The smart meter, with tighter clustering and better responsiveness, proves more suitable for accurate and consistent monitoring under such conditions.

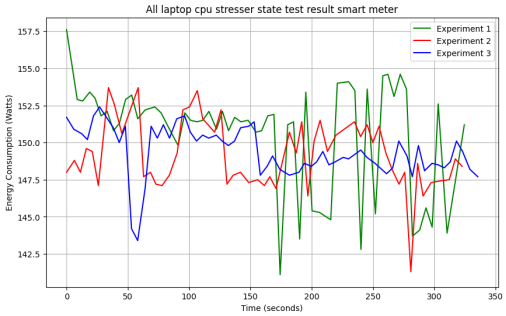


Figure 4.21: *Smart Meter laptop CPU stressor state experiment time series*

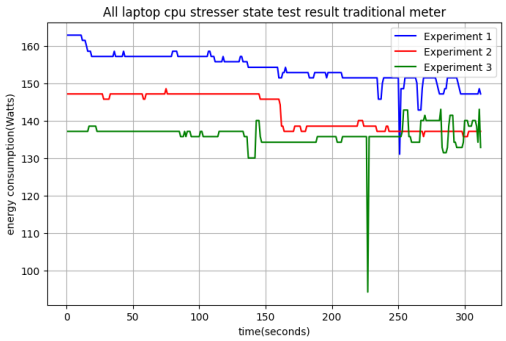


Figure 4.22: *Traditional Meter laptop CPU stressor state experiment time series*

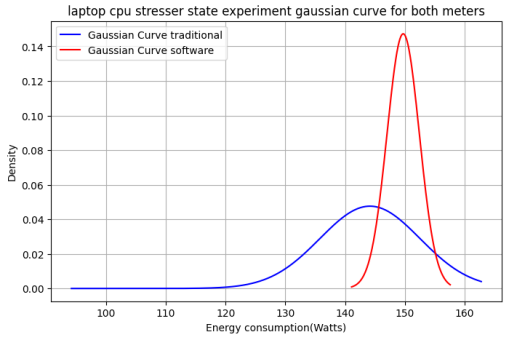


Figure 4.23: *Gaussian Distribution of both meters*

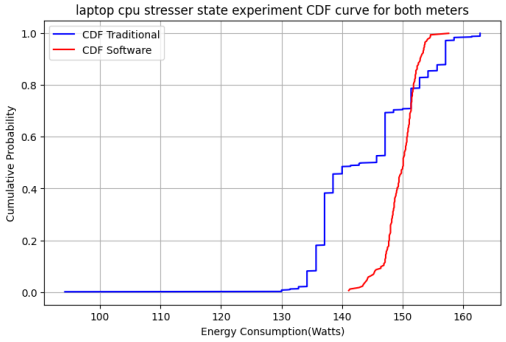


Figure 4.24: *CDF of both meters*

Figure 4.25: *Comparison of Smart and Traditional Meter Energy Measurements Across Different Analyses*

4.1.6 Summary of tests traditional meter vs smart meter

In this section, we discuss the observed pattern or trends from all tests and put them in the table above. From this we can summaries our findings and provide any implications for the meters.

Figure A.1 summarizes the performance of the smart and traditional meters across a range of tests with varying loads. The smart meter consistently captured real-time fluctuations and low-power activity with higher sensitivity, making it particularly effective for detecting fine-grained variations during idle or low-load conditions such as smart-watch and laptop idle tests. It also demonstrated better responsiveness during medium to high load scenarios, including CPU stress tests and gaming, where transient spikes were captured more accurately. In contrast, the traditional meter generally showed limited sampling resolution, evident through step-like patterns and a tendency to under-report or miss subtle variations, especially under low to medium load conditions. However, in scenarios where energy usage remained steady and within manufacturer-rated ranges, such as toaster testing, the traditional meter showed better alignment with expected values.

From a consumer perspective, the smart meter's ability to capture fine-grained, real-time data makes it ideal for households or individuals interested in monitoring their energy usage more precisely, identifying inefficiencies, and reducing energy bills. Its effectiveness in detecting low-load devices (e.g., smartwatches, idle laptops) supports growing smart home environments with many low-consumption electronics. However, its tendency to over-report in certain cases, such as the toaster test, suggests users must be aware of potential discrepancies in reported values. For businesses, especially those with energy efficiency goals or regulatory compliance needs,

the smart meter's high resolution and consistency provide valuable insights for audits, load management, and optimization. On the other hand, traditional meters, while less precise and consistent, may still offer value in stable, predictable usage contexts—such as industrial settings with consistent high-load equipment, where accuracy around baseline consumption is more critical than capturing short-term fluctuations. Their simplicity and longer lifespan may also reduce maintenance and replacement costs in such settings, making them more suitable for cost-conscious applications.

4.2 Research 2: Software meter versus software meter in Intel CPU

In this section, we will cover all the tests results as mentioned in section 3.2.2. In the test results we will only cover the gaussian distribution and CDF graphs as the time series graphs had too much results making it extremely hard for any meaningful deduction of data. Additionally, at the end, I will provide a graph comparing the root mean squared error(RMSE) of each experiment between the 2 software meters.

4.2.1 Laptop idle state results

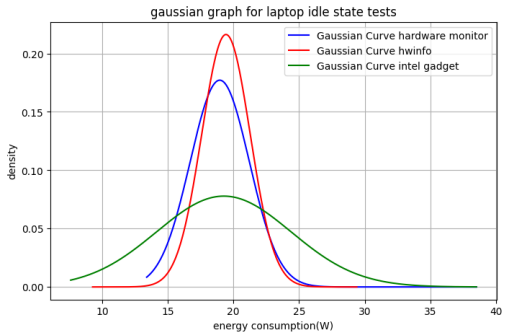


Figure 4.26: Software meters gaussian curve

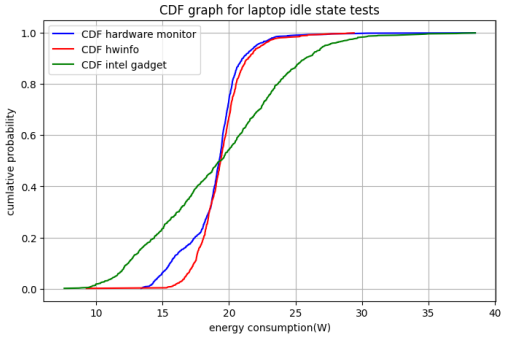


Figure 4.27: Software meters CDF curve

Figure 4.28: *Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU*

As shown in Figure 4.26, both HWiNFO and Open Hardware Monitor (OHM) produce Gaussian distributions centered around 18W to 20W, which closely align with the expected idle power range observed by Intel Power Gadget. However, HWiNFO demonstrates a much sharper and narrower peak compared to OHM, indicating more consistent and tightly clustered readings. OHM's broader distribution suggests greater variability and possibly reduced precision. When benchmarked against Intel Power Gadget(which shows a flatter and wider distribution) both tools appear less responsive to subtle power fluctuations. This may point to a limitation in sensitivity for both HWiNFO and OHM, but HWiNFO's higher measurement stability implies stronger internal consistency. From this, we can infer that HWiNFO is likely to be more repeatable under idle conditions, while OHM might slightly be more sensitive when observing energy consumption.

This interpretation is reinforced by the CDF in Figure 4.27. Intel Power Gadget's CDF rises more gradually over a broader range (extending up to 30W), indicating it captures a wide variety of idle power states, including subtle fluctuations and outliers. In contrast, both HWiNFO and OHM produce steeper, tightly clustered CDF curves concentrated between 18W and 25W, indicating

more uniform measurements. Notably, OHM displays a few outlier values that overlap with the higher end of Intel Power Gadget's range, suggesting that OHM may be more responsive to occasional spikes in idle power consumption. Additionally, OHM seems to be closer to Intel Power Gadget at the start in terms of distribution of the data. Meanwhile, HWiNFO's readings remain more narrowly distributed around the mean, reflecting higher consistency but potentially reduced sensitivity. This indicates that while HWiNFO may offer better repeatability in a stable idle state, it may also under-report transient power spikes. In contrast, OHM may sacrifice some consistency but better captures those momentary variations and seemingly distributes data closer to Intel Power Gadget at low load.

4.2.2 Laptop gaming state results

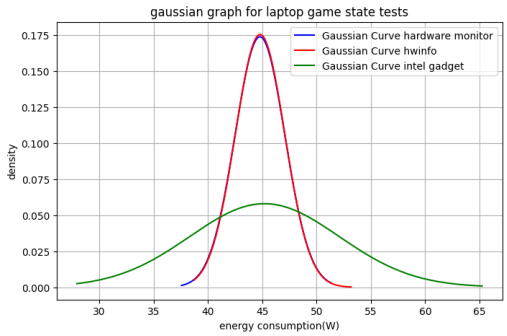


Figure 4.29: *Software meters gaussian curve*

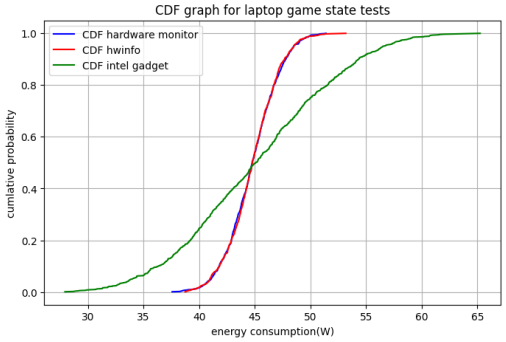


Figure 4.30: *Software meters CDF curve*

Figure 4.31: *Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU*

As shown in Figure 4.29, HWiNFO and Open Hardware Monitor both exhibit sharply peaked Gaussian distributions centered around 44W to 46W during the gaming test, indicating strong consistency in their readings. Both HWiNFO and OHM have extremely similar curves with OHM starting earlier and HWiNFO ending much later. However, both tools are limited in their range, capturing values only between 37W and 54W. In contrast, Intel Power Gadget shows a much wider and flatter distribution (30W to over 60W), revealing its ability to detect a greater range of fluctuations. This broader spectrum implies that HWiNFO and OHM may fail to register rapid or short-lived power changes that Intel Power Gadget can capture, which raises concerns about their suitability in dynamic workloads like gaming where responsiveness to transient activity is important.

Figure 4.30 reinforces this distinction. Both HWiNFO and OHM display steep, narrow CDF curves concentrated between 40W and 50W, indicating that most of their readings fall within a tight, consistent range. This highlights strong internal reliability but also points to limited sensitivity, especially when compared to Intel Power Gadget's more gradual CDF slope that spans from approximately 28W to 65W. The broader spread of Intel Power Gadget suggests it captures a more complete picture of transient power spikes and dips during gaming workloads. While HWiNFO and OHM yield nearly identical results (demonstrating agreement and consistency between the two) this also suggests that both may miss high or low outliers that the baseline detects. As such, despite their consistency, the narrower range of HWiNFO and OHM may result in under representation of dynamic power fluctuations, which are crucial during medium CPU intensive performance tasks like gaming. As observed, during low to medium CPU intensive tasks, both the meters tend to lack sensitivity with HWiNFO being much more consistent with mean readings and align more with the baseline mean readings.

4.2.3 Laptop CPU stressor with 5 processes and Idle threading results

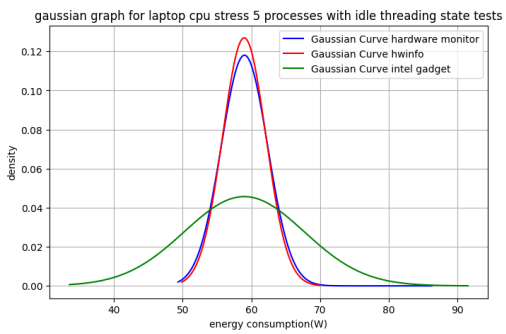


Figure 4.32: *Software meters gaussian curve*

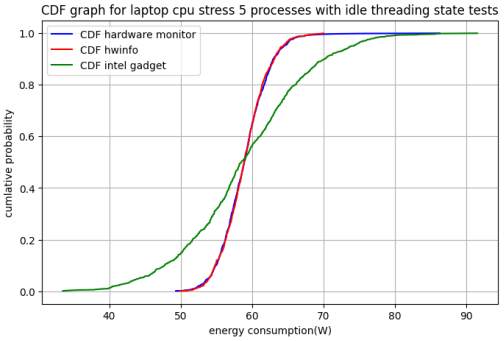


Figure 4.33: *Software meters CDF curve*

Figure 4.34: *Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU*

As shown in Figure 4.32, the Gaussian distributions of HWiNFO and Open Hardware Monitor are compared against the Intel Power Gadget baseline under light CPU stress conditions. Both HWiNFO and OHM show narrow, sharply peaked distributions centered around 58W to 60W, indicating strong internal consistency and agreement in their readings. HWiNFO exhibits the tightest distribution, suggesting it may be the most stable in terms of repeatability. However, Open Hardware Monitor does display one outlier extending toward 85W, introducing minor variability. In contrast, Intel Power Gadget shows a much broader curve, ranging from approximately 35W to over 90W, suggesting it is significantly more responsive to fluctuations and may be detecting subtle system-level events that the other meters smooth over or miss. While all three meters converge around the same mean value, suggesting general agreement, the broader span of Intel Power Gadget raises questions about the fidelity of HWiNFO and OHM in detecting outliers or short-term spikes.

This pattern is further reinforced in the CDF curves shown in Figure 4.33. HWiNFO and OHM again track each other very closely, with steep, narrow curves between 58W and 62W, confirming a strong level of consistency but also suggesting they are less responsive to outlier events. Intel Power Gadget, on the other hand, displays a much more gradual slope, covering a broader range from 35W to 90W, indicating greater sensitivity and real-time responsiveness. The alignment in the midpoint across all three meters suggests that while HWiNFO and OHM may offer reliable averages, Intel Power Gadget better captures the full spectrum of fluctuations. Additionally, OHM displays slightly more spread and outliers compared to HWiNFO, suggesting it is more sensitive to short-term spikes. In contrast, HWiNFO shows tighter clustering around the mean, favoring stability over responsiveness. This implies that while HWiNFO may provide more consistent averages, OHM may be better suited for capturing real-time fluctuations during medium CPU load, albeit with slightly less precision.

4.2.4 Laptop CPU stressor with 10 processes and low threading results

As shown in Figure 4.35, HWiNFO and Open Hardware Monitor (OHM) show closely aligned Gaussian distributions with sharp peaks between 63W and 66W, suggesting strong agreement and consistent performance under moderate CPU load. Both tools report readings mostly within a range of 52W to 79W. However, compared to Intel Power Gadget's broader curve, ranging from approximately 40W to beyond 90W, both software meters show reduced sensitivity to high or low extremes. This indicates that while HWiNFO and OHM effectively capture the

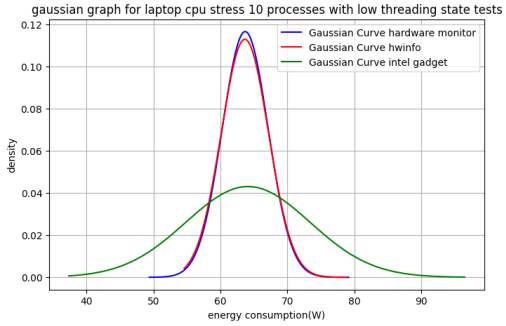


Figure 4.35: Software meters gaussian curve

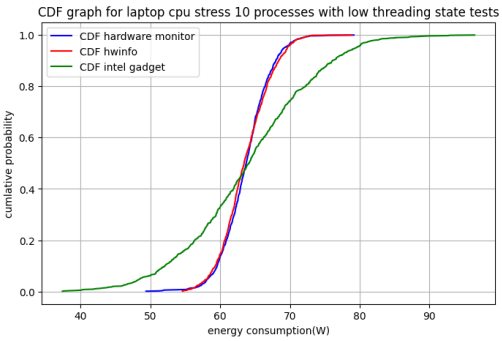


Figure 4.36: Software meters CDF curve

Figure 4.37: *Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU*

central trend, they may fail to register short-term spikes or dips that Intel Power Gadget picks up. The minimal spread difference between HWiNFO and OHM suggests similar behavior, but doesn't reveal which more closely matches the variability captured by the baseline.

The CDFs in Figure 4.36 reinforce the trend of high consistency from both HWiNFO and OHM under increased CPU load. Both exhibit steep rises between 60W and 70W, indicating tightly clustered values and limited deviation from the mean. While Intel Power Gadget captures a wider range (40W to 90W), reflecting its sensitivity to transient fluctuations, HWiNFO and OHM appear to smooth over such variability. In this scenario, OHM shows slightly better alignment with the mean, suggesting more stable output at moderate loads, though both meters remain extremely close in performance. The narrowing of distributions as CPU load increases implies that both tools become less responsive to spikes, under-representing dynamic fluctuations compared to Intel Power Gadget. This highlights a trade-off where HWiNFO and OHM offer repeatability but may lack fidelity under variable high-load conditions. Additionally, at high load we observe that both OHM and HWiNFO lack sensitivity as opposed to the previous medium to low load, since the distribution of data does not reach the points in which Intel Power Gadget reaches.

4.2.5 Laptop CPU stressor with 15 processes and normal threading results

As shown in Figures 4.38 and 4.39, HWiNFO and Open Hardware Monitor both produce narrow, sharply peaked Gaussian curves between 65W and 70W, with steep CDFs concentrated between 60W and 80W, thus demonstrating strong consistency in their readings. However, this narrow clustering highlights a potential drawback, where both meters may fail to register transient spikes or dips in energy consumption. In contrast, Intel Power Gadget presents a broader Gaussian curve and a gradual CDF rising from below 20W to nearly 140W, indicating greater responsiveness to real-time power fluctuations. HWiNFO's distribution aligns most closely with the mean of Intel Power Gadget, suggesting higher consistency around expected values. These results imply that while both HWiNFO and OHM are reliable for capturing average trends, they lack the sensitivity again to measure spikes as compared to Intel power gadget. However, we do observe that HWiNFO and OHM slightly deviate from each other as opposed to the other load results suggesting that at potentially higher loads both meters will have produce potentially a slight disparity in terms of results.

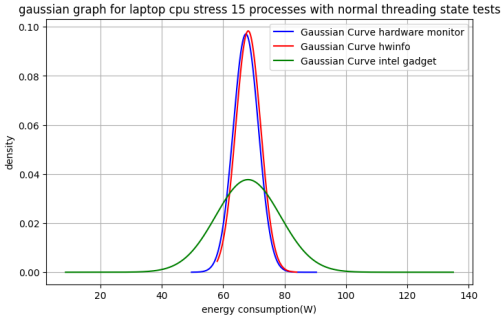


Figure 4.38: *Software meters gaussian curve*

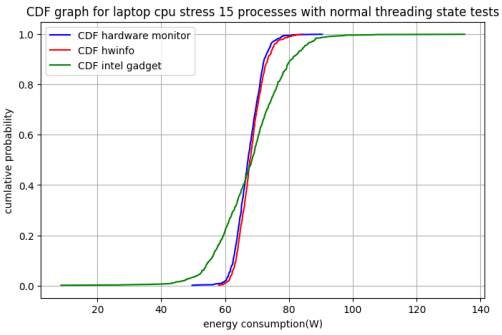


Figure 4.39: *Software meters CDF curve*

Figure 4.40: *Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU*

4.2.6 RMSE analysis for CPU tests

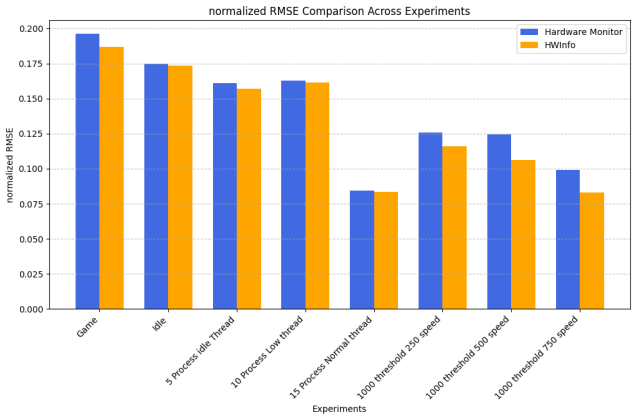


Figure 4.41: *Software meters RMSE bar chart*

As shown above there are multiple experiments in which we have not covered presented in the graph. Such experiments produced similar results to the previous experiments and thus explanations were not required. The graphs of such experiments are presented in the appendix.

As shown in Figure 4.41, the bar chart illustrates the normalized Root Mean Square Error (RMSE) across various CPU experiments, comparing HWiNFO and Open Hardware Monitor(OHM) against the Intel Power Gadget baseline. RMSE reflects how closely the software meters align with the baseline, where lower values indicate higher accuracy. Across all experiments, HWiNFO consistently demonstrates lower RMSE values than Open Hardware Monitor, suggesting it provides more accurate and reliable readings. The difference is especially notable in high-load scenarios like the "Game" state, where Open Hardware Monitor records the highest RMSE values, nearing 0.20. Additionally, we observe that HWiNFO tend to have a lower RMSE as the memory stressor tend to increase its rate whilst OHM remains the same suggesting memory constraints might potentially impact the meters performance. However, CPU load does not seem to correlate with RMSE in this case as we see a laptop 'idle' experiments having HWiNFO and OHM produce more RMSE as opposed to something like a extreme cpu stressor of 15 processes and normal threading. This trend reinforces that HWiNFO is generally more precise and stable, while Open Hardware Monitor tends to be slightly less aligned with the baseline, especially in

dynamic power usage situations. However, OHM is seen to observe spikes much better than HWiNFO, regardless OHM is still not as accurate as HWiNFO.

4.2.7 Laptop GPU game state test results

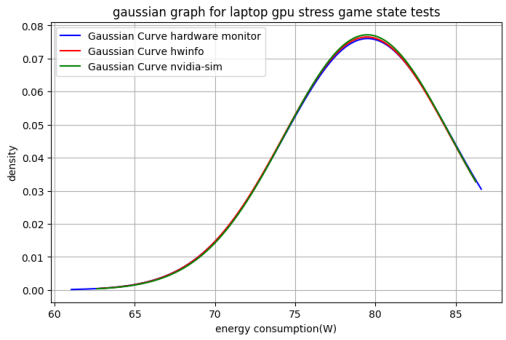


Figure 4.42: *Software meters gaussian curve*

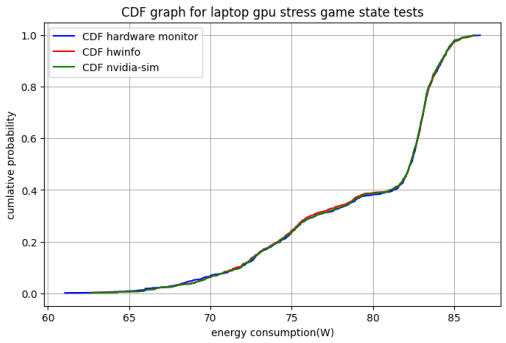


Figure 4.43: *Software meters CDF curve*

Figure 4.44: *Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU*

As shown in Figures 4.42 and 4.43, HWiNFO and Open Hardware Monitor (OHM) closely align with the Nvidia-SMI baseline during the GPU gaming workload, with all three Gaussian distributions centered tightly around 79W to 80W. This alignment suggests that both software meters are capable of delivering accurate average power readings under steady GPU loads. However, while the curves appear nearly identical at the mean, a deeper look shows that HWiNFO tracks Nvidia-SMI more precisely, with a slightly narrower distribution, indicating stronger consistency. OHM, although generally accurate, shows a marginally broader spread, implying slightly higher variability in its readings. This subtle difference is reinforced in the CDF, where HWiNFO follows the Nvidia-SMI curve more tightly, especially in the middle and upper ranges, while OHM diverges slightly. These results suggest that HWiNFO may offer more stable and accurate measurements during GPU-bound scenarios, while OHM, though comparable, may be slightly less precise under identical conditions. Unlike the CPU tests, these tests have each meter distribute results near identical to each other suggesting consistency across the meters when measuring GPU energy consumption.

4.2.8 Laptop GPU heavy load stressor test results

As shown in Figure 4.45, all three meters exhibit nearly identical Gaussian distributions centered around 94W, suggesting strong agreement in average power consumption under GPU stress. However, OHM displays a slightly sharper peak, indicating a narrower spread and potentially reduced responsiveness to small fluctuations, which may suggest over-concentration of values near the mean. In contrast, HWiNFO more closely mirrors the broader shape of the Nvidia-SMI distribution, indicating better alignment with the baseline in terms of capturing variability. This distinction becomes clearer in the CDF plot (Figure 4.46), where HWiNFO's curve tracks Nvidia-SMI more consistently across the full power range suggesting more accuracy, while OHM rises more steeply, reflecting less diversity in the readings. These findings imply that although both meters are reasonably accurate under sustained GPU load, HWiNFO provide slightly more accurate readings of power consumption compared to OHM in higher GPU loads.

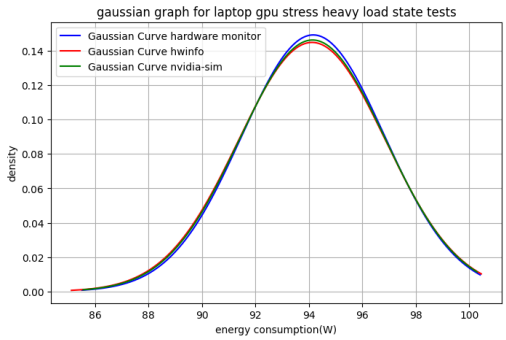


Figure 4.45: Software meters gaussian curve

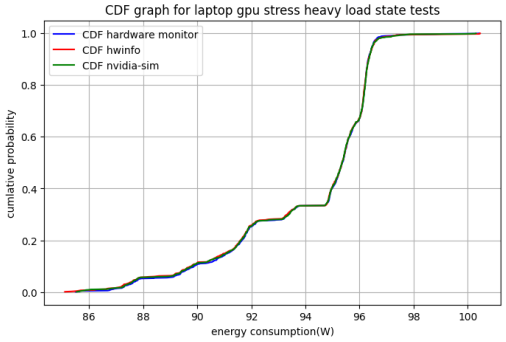


Figure 4.46: Software meters CDF curve

Figure 4.47: Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU

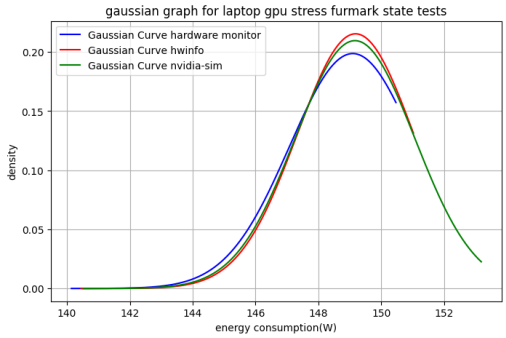


Figure 4.48: Software meters gaussian curve

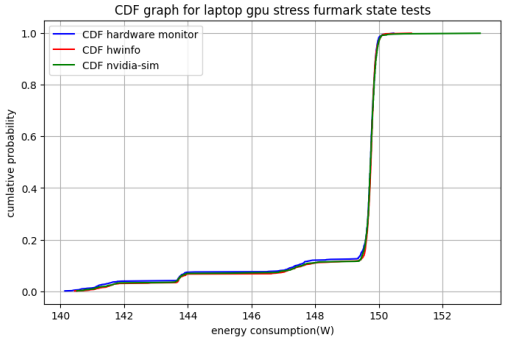


Figure 4.49: Software meters CDF curve

Figure 4.50: Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU

4.2.9 Laptop GPU Furmark stressor test results

As shown in Figures 4.48 and 4.49, all three meters exhibit closely aligned energy consumption patterns under high GPU load, with Gaussian curves peaking near 149–150W. However, subtle differences in distribution reveal variation in how each tool captures frequency near the mean. HWiNFO displays the sharpest and tallest peak, indicating tight clustering around the average and suggesting high internal consistency. Nvidia-SMI, while flatter, provides a more balanced spread, likely reflecting its hardware-level precision. OHM, in contrast, shows a broader and slightly offset distribution, under representing the central peak while over-representing values between 144W and 147W. This discrepancy is echoed in the CDF plot, where OHM accumulates readings more rapidly in the lower range, diverging from the smoother progression of HWiNFO and Nvidia-SMI. These patterns suggest that although both software meters are generally reliable, OHM may slightly under predict or misrepresent peak energy values under extreme GPU load, whereas HWiNFO more closely mirrors the baseline, offering improved fidelity. Notably, neither software meter fully captures the extended dynamic range reflected by Nvidia-SMI, especially at peak values. While HWiNFO comes closer, it still under-reports some high-end spikes, indicating both software meters may struggle to track precise energy consumption under extreme GPU loads. Despite this, HWiNFO's closer alignment with the baseline reinforces its relative suitability for high-load scenarios, whereas OHM may be less reliable in such demanding

environments. However, both the meters tend to strive slightly away from the baseline, suggesting that at extremely high GPU loads the meters may tend to misrepresent the energy consumption.

4.2.10 RMSE analysis for GPU tests

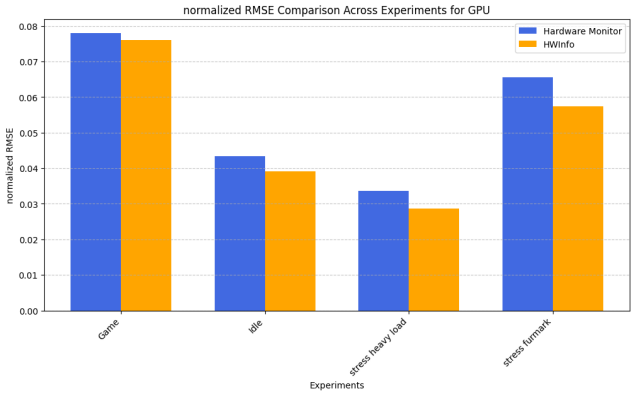


Figure 4.51: *Software meters gaussian curve*

As shown in Figure 4.51, the normalized RMSE values across various GPU workloads(ranging from idle and gaming to intensive stress scenarios), highlight key differences in how well HWiNFO and Open Hardware Monitor (OHM) align with the NVIDIA-SMI baseline. HWiNFO consistently produces lower RMSE values across all tests, suggesting better accuracy and stronger agreement with the hardware-level benchmark. This is especially significant under high-load conditions, such as the Furmark stress test, where OHM's RMSE sharply increases, indicating a noticeable drop in precision. While both tools perform similarly under low or moderate workloads, OHM's growing deviation under extreme stress suggests limitations in its ability to handle volatile or peak power readings. In contrast, HWiNFO maintains more stable accuracy, even under demanding conditions, reinforcing its suitability for tracking real-time GPU consumption in high-performance environments. These findings imply that OHM may be adequate for general use but less reliable in scenarios requiring precise power monitoring, especially when systems operate near their performance limits. Unlike the CPU test results, these results imply that at higher GPU stress loads the meters tend to make mistakes, however HWiNFO still proved to be much more accurate.

4.3 Research 3: Software meter versus software meter in AMD CPU

In this section, we will cover all the tests results as mentioned in section 3.2.2, with an addition of testing the experiments on a different CPU type. This section will provide graphs and an analysis of all the graphs similar to the previous section.

4.3.1 Laptop idle state results

As shown in Figure 4.52 and 4.53, the Gaussian and cumulative distribution function (CDF) plots illustrate the energy consumption patterns in the idle state, with uProf acting as the baseline. In the Gaussian curve, while HWiNFO shows a sharper and denser peak around 6.9W–7W, it slightly overshoots the baseline distribution of uProf. Interestingly, Open Hardware Monitor (OHM) aligns its peak frequency more closely to that of uProf, even if its curve is broader. This suggests that although HWiNFO is more consistent in terms of tighter spread, OHM better reflects

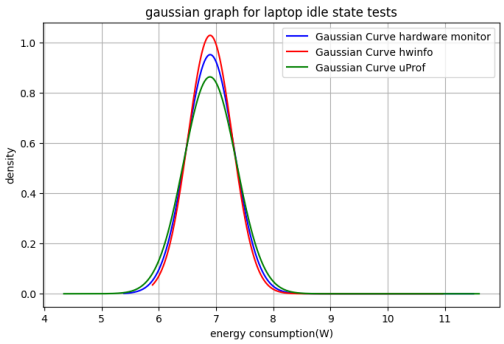


Figure 4.52: Software meters gaussian curve

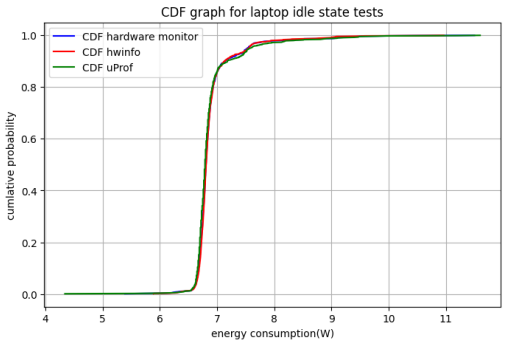


Figure 4.53: Software meters CDF curve

Figure 4.54: Comparison of Software Meters Energy Measurements Across Different Analyses in AMD CPU

uProf's actual central tendency. This pattern is reinforced in the CDF plot, where HWiNFO's curve tracks OHM more closely than it does uProf, especially near the upper tail where uProf begins to diverge slightly beneath both the HWiNFO and OHM curves. However, uProf shows a much wider spread in both its Gaussian and CDF curves, indicating greater sensitivity to transient fluctuations that HWiNFO and OHM failed to capture. This divergence implies that while both software meters provide stable and repeatable readings, they may under-report subtle variations at low loads, potentially due to limited resolution or smoothing of minor spikes. Thus, although all three meters appear consistent, uProf's broader range highlights its superior responsiveness to low-level fluctuations, raising questions about the sensitivity of HWiNFO and OHM in idle-state monitoring.

4.3.2 Laptop game state results

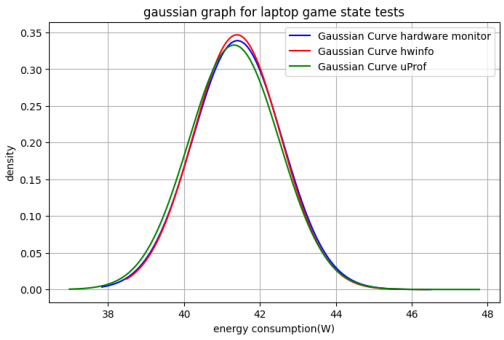


Figure 4.55: Software meters gaussian curve

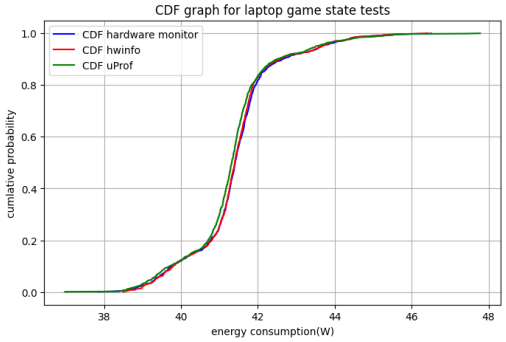


Figure 4.56: Software meters CDF curve

Figure 4.57: Comparison of Software Meters Energy Measurements Across Different Analyses in AMD CPU

As shown in Figures 4.55 and 4.56, the Gaussian and CDF plots illustrate the energy consumption behavior of HWiNFO and Open Hardware Monitor (OHM) relative to uProf during gaming. All three meters peak around 41W, indicating strong agreement in average consumption. However, HWiNFO shows a tighter and more sharply defined peak, suggesting more consistent measurements, while OHM's broader curve indicates slightly greater variability. Notably, OHM's distribution leans marginally to the right of uProf, implying a tendency to over-predict energy

usage during gaming scenarios. This pattern is echoed in the CDF plot, where HWiNFO more closely tracks uProf's curve throughout the distribution, particularly beyond the 0.6 probability mark. OHM begins to deviate upward, accumulating readings at a faster rate, which may suggest inflated mid-range values. Furthermore, the positioning of uProf above both software meters between the 0.2 and 0.8 cumulative probability range suggests that HWiNFO and OHM may be consistently reporting higher-than-actual values in this band. Additionally, uProf captures a broader range of readings, reaching up to 48W, suggesting that both software meters may miss short-lived power spikes due to reduced sensitivity. Overall, while both tools provide reasonable estimates, HWiNFO offers slightly better alignment with the baseline, whereas OHM appears more prone to over-prediction and under-sensitivity to transient peaks. In the case of a medium load, we observe slight deviations in the readings across the meters, with HWiNFO aligning more closely to the baseline. This suggests that as energy consumption increases, some meters may begin to exhibit inaccuracies or drift from the true value.

4.3.3 Laptop CPU stressor with 5 processes and idle threading results

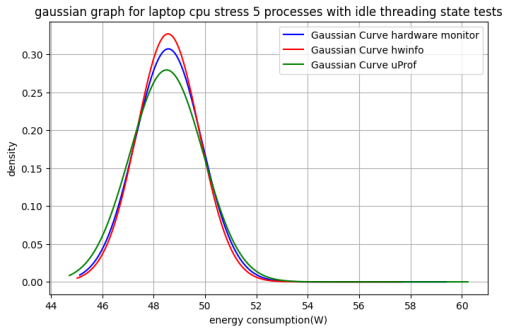


Figure 4.58: *Software meters gaussian curve*

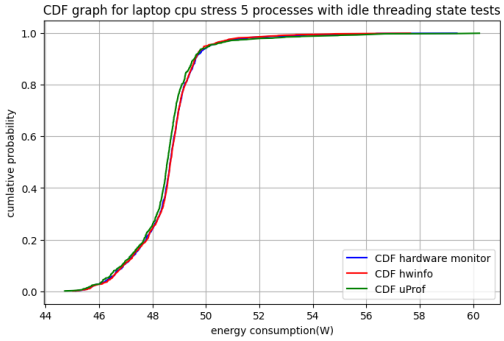


Figure 4.59: *Software meters CDF curve*

Figure 4.60: *Comparison of Software Meters Energy Measurements Across Different Analyses in AMD CPU*

As shown in Figure 4.58 and 4.59, the Gaussian and CDF plots compare HWiNFO and Open Hardware Monitor (OHM) against uProf during a low-load CPU stress test. All three tools peak around 48.5W, indicating similar average consumption. HWiNFO displays the sharpest and narrowest Gaussian curve, suggesting it produces highly repeatable readings but may filter out short-term variability. OHM shows a slightly broader curve, reflecting more variance, while uProf's wide distribution extending beyond 60W indicates greater sensitivity to momentary power spikes. This difference is reinforced in the CDF plot, where uProf begins its rise earlier and progresses more gradually, capturing a broader spectrum of values. Notably, uProf's CDF sits below the curves of HWiNFO and OHM, implying that both software meters tend to overestimate consumption relative to the baseline. Notably, OHM ends closer to uProf's upper tail than HWiNFO, suggesting that while it is less consistent than HWiNFO, it is more responsive to power deviations. Overall, this implies HWiNFO is ideal for stable average tracking, whereas OHM may better reflect subtle power shifts under low CPU stress, though still not matching uProf's full dynamic sensitivity. However, both meters still slightly overestimate values at low to medium CPU load.

4.3.4 Laptop CPU stressor with 10 processes and low threading results

As shown in Figures 4.61 and 4.62, the Gaussian and CDF plots compare HWiNFO and Open Hardware Monitor (OHM) against uProf under a high CPU load. All three meters peak around

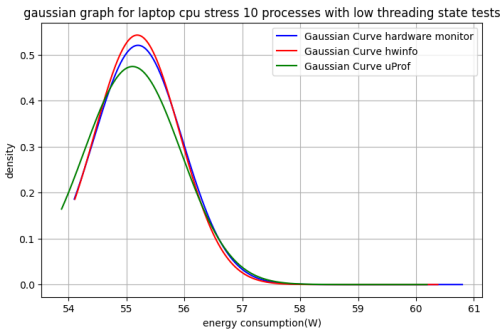


Figure 4.61: Software meters gaussian curve

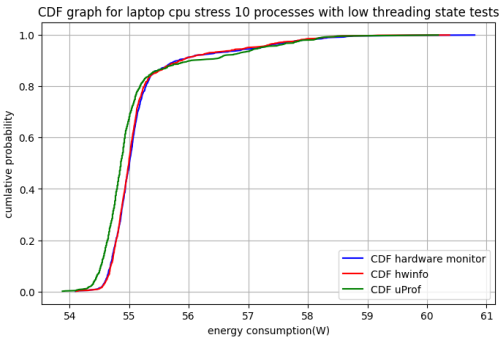


Figure 4.62: Software meters CDF curve

Figure 4.63: Comparison of Software Meters Energy Measurements Across Different Analyses in AMD CPU

55W, indicating broad agreement on the mean energy usage. However, HWiNFO aligns most closely with uProf in terms of peak position, while OHM peaks slightly later and exhibits a broader spread, suggesting slightly higher variability. Uprof has a distribution of up until 60.1W. In contrast, OHM and HWiNFO slightly over-predict relative to the baseline, with OHM nearing 61W and HWiNFO just below 60W. This is further reflected in the CDF plot, where uProf begins its rise way earlier and progresses more gradually, capturing more subtle transitions in power use. Notably, since uProf curve is above the other 2 meters, this suggest the other 2 meters were over predicting values again. While HWiNFO have matching means with uProf suggesting accuracy and consistency, both HWiNFO and OHM tend to slightly over predict values and spikes at much higher CPU load, thus questioning their reliability in such environments. Unlike the other load testers, these results show that as the CPU stressor becomes increasingly demanding, both HWiNFO and Open Hardware Monitor (OHM) gradually diverge from the baseline.

4.3.5 Laptop CPU stressor with 15 processes and normal threading results

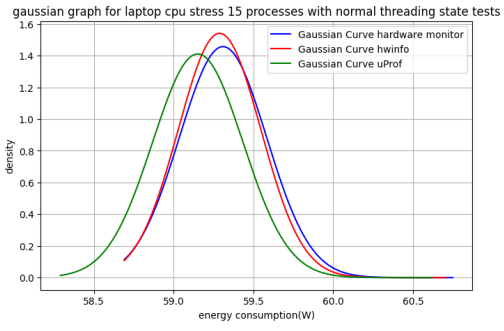


Figure 4.64: Software meters gaussian curve

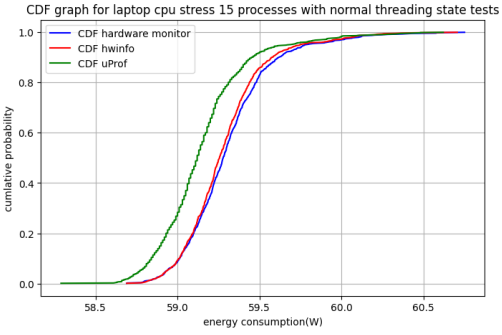


Figure 4.65: Software meters CDF curve

Figure 4.66: Comparison of Software Meters Energy Measurements Across Different Analyses in AMD CPU

As shown in Figure 4.64 and 4.65, the Gaussian and CDF plots illustrate the energy consumption behavior of HWiNFO and Open Hardware Monitor (OHM) compared to the uProf baseline during a high-load CPU stress test. All three Gaussian curves are tightly clustered and peak

around 59.2W to 59.4W, showing strong agreement in central tendency. However, both HWiNFO and OHM slightly overshoot uProf's peak, indicating they may slightly overestimate the mean energy consumption. Among the software meters, HWiNFO presents the sharpest and tallest peak, suggesting highly concentrated readings, while OHM shows a slightly broader distribution. As observed in previous tests, both HWiNFO and OHM appear to over-distribute values beyond uProf's maximum range(extending slightly above 60.6W), with HWiNFO being much closer to uProf. This trend is further supported by the CDF plot, where uProf's curve begins earlier (around 57.7W) and rises more gradually, indicating sensitivity to a wider spread of lower values. HWiNFO's CDF remains closer to uProf's trajectory than OHM's, which diverges more noticeably. However, OHM and HWiNFO CDF curves have a large gap between uProf, suggesting overestimation of results. This increasing divergence at higher loads suggests that software meters may become less accurate under extreme conditions, this is as proven in the experiment above and here.

4.3.6 RMSE of CPU results

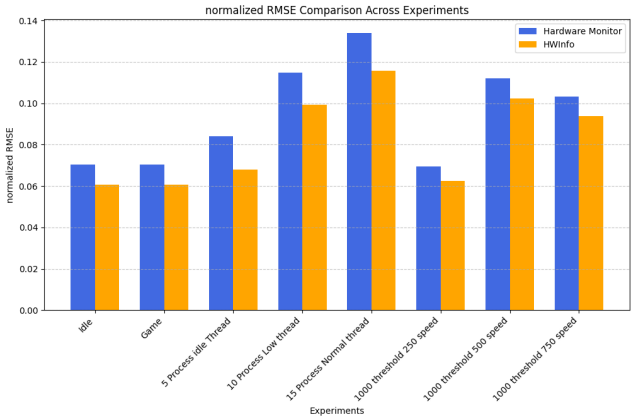


Figure 4.67: *Software meters RMSE bar chart*

As mentioned in the section earlier, there are tests in which I have not analyzed as the results seemed to be the same and as such I will simply display their RMSE values for comparison. If you would like to view their graph, it is presented in the appendix.

The bar chart in Figure 4.67 presents the normalized RMSE values for HWiNFO and Open Hardware Monitor (OHM) across various AMD-based experiments, providing a comparative view of how closely each meter aligns with the uProf baseline. HWiNFO consistently reports lower RMSE values than OHM in every experiment, indicating better accuracy. In the "idle" and "game" states, both meters perform best, with HWiNFO recording RMSE values of approximately 0.061 and 0.060, respectively, while OHM follows closely at 0.070 for both cases. However, the gap widens in high-load CPU stress tests. For example, in the "15 processes normal threading" experiment, OHM reaches the highest RMSE at approximately 0.13, while HWiNFO remains significantly lower at around 0.117. During memory stressor tests, both meters show increased RMSE values with rising threshold speeds, though the difference between them narrows slightly at the "1000 threshold 750 speed" condition, OHM stands at approximately 0.103 while HWiNFO is slightly lower at 0.094. These results suggest that while both meters are reasonably reliable under light workloads, HWiNFO consistently delivers more precise and stable measurements across both moderate and extreme CPU conditions. Additionally, under high loads as observed above, the meters tend to over predict results which is a consideration consumers must take when utilizing such a product.

4.3.7 Laptop GPU game test results

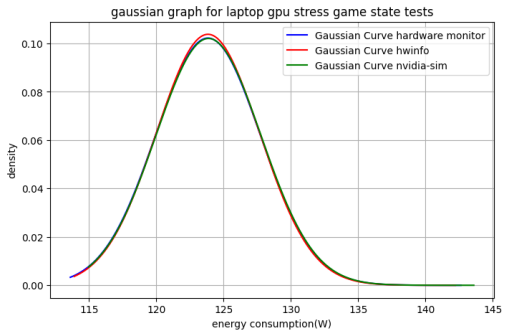


Figure 4.68: *Software meters gaussian curve*

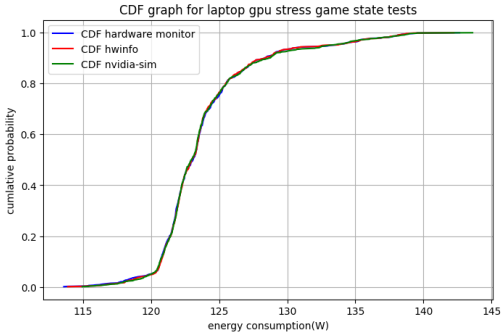


Figure 4.69: *Software meters CDF curve*

Figure 4.70: *Comparison of Software Meters Energy Measurements Across Different Analyses in AMD CPU*

As shown in Figures 4.68 and 4.69, the Gaussian and CDF plots compare the energy consumption readings of HWiNFO and Open Hardware Monitor (OHM) against the Nvidia-SMI baseline during a gaming state. The Gaussian curves for all three tools show strong alignment, peaking around 124W–125W, with HWiNFO showing a slightly sharper peak. This suggests that HWiNFO recorded more concentrated readings, while OHM and Nvidia-SMI captured slightly broader distributions. Importantly, the mean distributions are nearly identical across the three meters, highlighting consistent behavior. The CDF plot reinforces this, showing all three curves progressing closely from 114W to around 144W, with OHM starting first and Nvidia-SMI ending last. There is near-perfect overlap among the meters, with very minimal deviation at the tails, indicating excellent agreement in both cumulative probabilities and data spread. However, the slightly wider distribution from Nvidia-SMI may indicate higher sensitivity to transient power changes that software tools may smooth out. Overall, whilst OHM seemed to be closer to the baseline results, the 2 meters still are not accurately predicting spikes which again questions their sensitivity. Additionally, under GPU load tests, at moderate to near high load both the meters tend to perform and produce accurate results unlike the CPU stressor results.

4.3.8 Laptop GPU Furmark stress test results

As shown in Figures 4.71 and 4.72, the Gaussian and CDF plots reveal how HWiNFO and Open Hardware Monitor (OHM) perform relative to the Nvidia-SMI baseline under extreme GPU load using Furmark. All three Gaussian curves peak near 141W, indicating strong agreement in average power consumption. HWiNFO exhibits the sharpest peak, suggesting highly consistent readings with minimal internal variation, while OHM shows a slightly broader distribution, reflecting marginally greater variability. Nvidia-SMI, serving as the hardware-level baseline, presents a wider spread, beginning at approximately 131W and extending to 154W(broader than both software meters), which start around 134–136W and cap at roughly 150W. This suggests that while HWiNFO and OHM produce stable measurements, they may slightly under-report spikes at the lower and upper ends. In the CDF plot, the curves for HWiNFO and OHM track Nvidia-SMI very closely throughout the 134W–150W range, indicating excellent alignment across most of the workload. However, the slightly earlier rise and longer tail of the Nvidia-SMI curve again highlight its superior sensitivity to edge-case values. Overall, both software meters perform reliably under heavy GPU loads, with HWiNFO offering more consistency and OHM capturing a slightly broader spread, but both fall just short of matching the full dynamic range of Nvidia-SMI which again questions their sensitivity when measuring energy consumption in

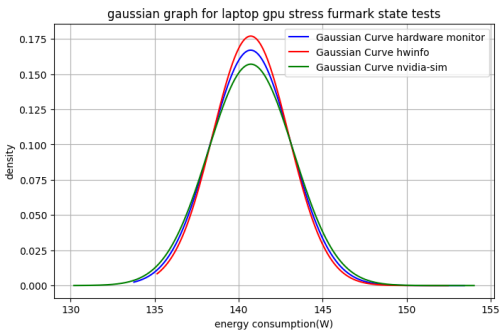


Figure 4.71: *Software meters gaussian curve*

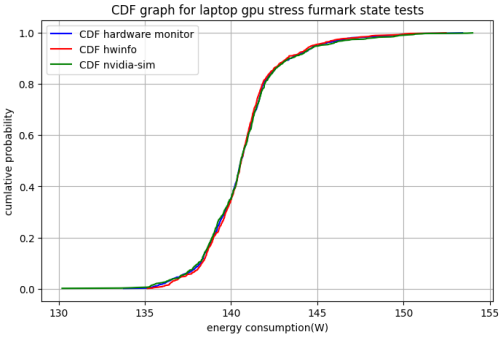


Figure 4.72: *Software meters CDF curve*

Figure 4.73: *Comparison of Software Meters Energy Measurements Across Different Analyses in AMD CPU*

high GPU utilization environment.

4.3.9 RMSE of GPU results

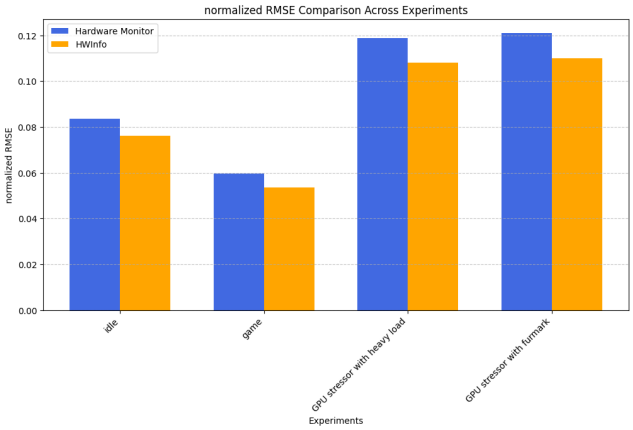


Figure 4.74: *Software meters RMSE bar chart*

As mentioned in the section earlier, there are tests in which I have not analyzed as the results seemed to be the same and as such I will simply display their RMSE values for comparison. If you would like to view their graph, it is presented in the appendix.

The bar chart in Figure 4.74 presents the normalized RMSE values for HWiNFO and Open Hardware Monitor (OHM) across different GPU-related experiments using Nvidia-SMI as the baseline. As observed, HWiNFO consistently produces lower RMSE values than OHM in all test scenarios, highlighting its superior accuracy in aligning with the baseline. During the idle state, HWiNFO records an RMSE of approximately 0.076 compared to OHM's 0.083, and during the gaming state, the gap narrows slightly with HWiNFO at about 0.054 and OHM at 0.060. The difference becomes more pronounced under stress conditions. For the "GPU stressor with heavy load" experiment, OHM reaches a peak RMSE of around 0.119, while HWiNFO stays lower at about 0.109. Similarly, in the "GPU stressor with Furmark" test, OHM again shows the highest RMSE value of approximately 0.122, while HWiNFO maintains better alignment at roughly 0.110. This trend suggests that while both meters perform adequately in less demanding

conditions, OHM struggles with accuracy as workload intensity increases. Additionally, we observe that as the GPU stress load becomes much higher, both meters tend to have higher RMSE, thus questioning their performance at higher GPU load environments. Although HWiNFO has much better accuracy, both the meters struggled observing spikes in higher load environments, thus again questioning their sensitivity. For consumers, particularly those relying on software meters for gaming, machine learning or anything related to high GPU usage, this implies HWiNFO may provide more accurate feedback for system tuning and energy profiling, thus making it a more reliable choice in scenarios demanding precision. Additionally, just like in Intel CPU, OHM and HWiNFO had extremely close results to the baseline in the GPU tests.

4.4 Research 4: Software meter versus Software meter usability testing results

As previously outlined, understanding the usability of energy monitoring tools is just as important as evaluating their accuracy, especially for end users who may not have technical expertise. As outlined in Section 3.2.3, usability insights were collected from a total of 20 participants. Each participant completed a set of fundamental energy monitoring tasks using both software meters. Following this, they responded to a 12-question Likert-scale survey, designed to evaluate four key usability dimensions: ease of use, perceived usefulness, intention to use, and overall user satisfaction. In the sections that follow, we begin by analyzing the quantitative results from the Likert scale responses. This is followed by an evaluation of the qualitative feedback highlighting usability issues encountered by participants. Finally, we present a comparative summary of the two meters based on overall user experience.

4.4.1 Quantitative analysis

	Likert scale questions	TAM
Q1	I intend to use this software regularly when monitoring system performance	Intention to use
Q2	Given the choice, I would recommend this software to others for hardware monitoring.	Intention to use
Q3	I thought the software was easy to use.	Perceived Ease of Use
Q4	I did need external help to set up the software.	Perceived Ease of Use
Q5	Once the software is set up, I found it easy to use for my energy monitoring tasks.	Perceived Ease of Use
Q6	Most people would learn to use this software very quickly.	Perceived Ease of Use
Q7	I found the software's interface to be user-friendly.	Perceived Ease of Use
Q8	The software helped me accomplish my hardware monitoring tasks effectively.	Perceived usefulness
Q9	This software offered solutions that other similar tools do not provide.	Perceived usefulness
Q10	I found the information provided by the software to be accurate and reliable.	Perceived usefulness
Q10	I am happy with the software as it is.	User satisfaction

Table 4.1: *The Likert scale questions and the TAM dimension associated with such questions.*

The bar charts in Figures 4.75 and 4.76 display the average Likert-scale scores for HWiNFO and Open Hardware Monitor (OHM), mapped against the four key TAM dimensions: Intention to Use, Perceived Ease of Use, Perceived Usefulness, and User Satisfaction. When comparing the two charts, it is evident that HWiNFO outperforms OHM across nearly every category,

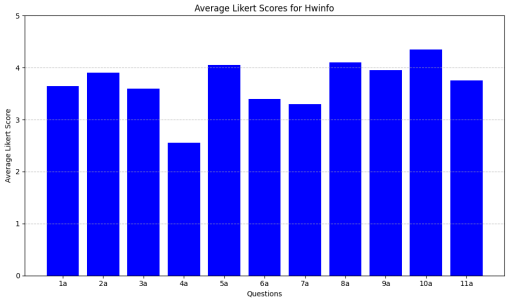


Figure 4.75: *HWiNFO survey Likert scale results*

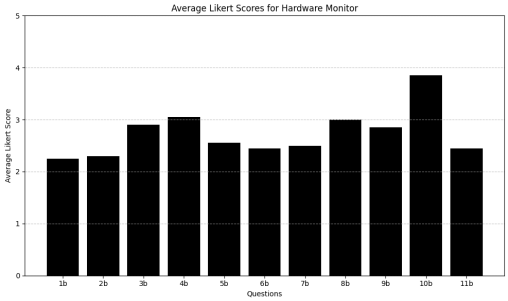


Figure 4.76: *Open hardware monitor survey Likert scale results*

Figure 4.77: *Comparison of Software Meters Usability*

particularly in areas related to perceived usefulness and user satisfaction. Questions Q1 and Q2, which reflect "Intention to Use," show HWiNFO scoring close to 4, while OHM lags significantly behind, scoring near 2.2–2.3. This suggests that users are more willing to adopt and recommend HWiNFO, possibly due to a more positive overall experience.

In the "Perceived Ease of Use" category (Q3–Q7), HWiNFO again demonstrates stronger performance. Users reported fewer barriers to using the software, especially once it was set up, as indicated by Q5a scoring above 4.0. Meanwhile, OHM scored considerably lower across all ease-of-use questions, typically falling in the 2.5 to 3.1 range. This points to a usability gap where OHM may be harder for users to navigate, especially those without a technical background. This is further validated by Q4, which measures reliance on external help, where OHM users indicated a higher need for assistance, undermining its perceived ease of adoption. This also suggests, new users will struggle to use OHM as opposed to HWiNFO given the results.

The contrast is even more pronounced in the "Perceived Usefulness" dimension (Q8–Q10). HWiNFO receives some of its highest ratings here, particularly in Q10a (accuracy and reliability), which peaks around 4.3–4.4. This indicates users felt confident in the data provided by HWiNFO and found it effective for their monitoring tasks. OHM, in contrast, shows lower scores in both usefulness and uniqueness of features, with only Q10b nearing 3.8. This disparity suggests that while OHM may be functional, it lacks the user-perceived value and reliability found in HWiNFO. Lastly, "User Satisfaction" (Q11) reinforces these findings. HWiNFO scores around 3.8, indicating a generally positive user experience, whereas OHM scores below 2.5, pointing to broader dissatisfaction. This overall trend implies that HWiNFO is not only more accurate and easier to use, but also leaves users with a greater sense of satisfaction, which is crucial for long-term adoption.

Overall, this clearly shows HWiNFO is more usable and user friendly across all the TAM dimensions. Its combination of intuitive design, perceived utility, and consistent reliability makes it the preferred choice among users, both technical and non-technical. OHM, while functional, may require interface improvements and better setup guide to close the usability gap.

4.4.2 Qualitative analysis

In this section we will describe any qualitative issues that users faced while performing the given tasks.

For HWiNFO, the majority of participants were able to complete all six tasks with minimal difficulty, indicating that the software is generally functional and discoverable. However, a few users faced challenges completing tasks related to logging system data (Task 6), locating audio

devices (Task 5), and identifying CPU voltage/energy (Task 4). Users attributed these difficulties not to the absence of functionality but rather to a cluttered and overly technical interface, where relevant features were difficult to locate. Some non-technical users also cited confusion around terminology, which reflects a potential mismatch between HWiNFO's interface design and the expectations of less experienced users. Despite these shortcomings, HWiNFO was still perceived as comprehensive and relatively intuitive once users became familiar with the layout, highlighting that the learning curve may be steeper, but ultimately worth it due to its feature set.

In contrast, Open Hardware Monitor (OHM) showed more pronounced usability limitations. All participants reported being unable to complete at least one task, particularly Task 5 (identify audio devices) and Task 6 (log system data). This failure was not due to user oversight but due to OHM lacking those features entirely, revealing fundamental limitations in functionality. Moreover, some users could not locate basic readings like CPU temperature, despite the feature being present, this reflects issues not only in feature availability but also in UI discoverability. Users described the interface as "terrible" and outdated, suggesting frustration with navigation and visual design. Additionally, users faced barriers during the installation process, noting the lack of a user-friendly website, minimal setup guidance, and no official installer, unlike HWiNFO, which offers a structured setup process and downloadable installer.

Critically, these observations suggest that while HWiNFO may initially overwhelm some users with a dense UI, its broader capabilities and more polished setup experience contribute positively to usability in the long term. Conversely, OHM's lack of essential monitoring features and unintuitive interface design significantly diminish its practicality, especially for users seeking a comprehensive monitoring tool. This discrepancy is also reflected in the Likert scale responses, where OHM scored lower across multiple TAM dimensions such as perceived usefulness and user satisfaction. Ultimately, HWiNFO's advantage lies in its functional depth combined with moderate learnability, while OHM suffers from limited scope and usability friction, both in usage and onboarding.

4.4.3 Summary

Based on the TAM-based Likert analysis and qualitative task performance, HWiNFO clearly outperforms Open Hardware Monitor (OHM) in nearly all usability dimensions. HWiNFO demonstrated higher average scores across the TAM categories, indicating a more intuitive and rewarding user experience. It allowed most users, including those with limited technical backgrounds, to complete monitoring tasks effectively, even if some struggled initially with navigating the dense interface. Importantly, all core features required for monitoring were present and accessible, affirming its status as a comprehensive and functional tool.

In contrast, OHM exhibited fundamental shortcomings. Users consistently reported difficulties completing essential tasks due to missing features (e.g., audio device detection and data logging) and poor UI discoverability. Even where features existed, such as temperature monitoring, OHM's interface design impeded their use. This was further compounded by a frustrating installation process, which lacked user guidance—making the tool inaccessible for non-technical users. These issues contributed to OHM's lower TAM scores, particularly in perceived usefulness and intention to use.

In summary, HWiNFO offers a more complete and user-friendly experience, especially in real-world monitoring contexts, while OHM falls short in both functionality and usability. These findings suggest that for both casual users and technical audiences, HWiNFO is the more practical and reliable choice, whereas OHM would require significant UI and feature enhancements to match the level of usability and adoption readiness of HWiNFO.

5 Conclusion

This chapter will provide a summary of the whole research paper alongside a reflection of our work and finally we will end this paper off with future works that could take place given this baseline of a paper.

5.1 Summary

This study set out to evaluate and compare the performance and usability of energy consumption measurement tools across both hardware (smart vs. traditional meters) and software (HWiNFO vs. Open Hardware Monitor) platforms. In the hardware-based experiments, smart meters consistently outperformed traditional meters across various appliance tests ranging from low-load (e.g., smartwatches) to high-load (e.g., kettles). Smart meters provided more stable and accurate readings, exhibited tighter Gaussian distributions, and tracked real-time fluctuations more effectively in both CDF and time-series plots. In contrast, traditional meters struggled particularly under low-load conditions, often failing to detect small consumption and produced step-like, less responsive data, undermining their reliability. Although traditional meters sometimes aligned well under fixed load scenarios like toaster testing, they lacked the granularity and sensitivity needed for modern energy monitoring use cases.

Software meter analysis further validated the superior precision of HWiNFO compared to Open Hardware Monitor (OHM). Across CPU and GPU experiments using Intel Power Gadget, Nvidia-SMI, and uProf as baselines, HWiNFO consistently yielded lower RMSE values, demonstrating closer alignment to hardware baselines. It also tracked dynamic changes better, especially under high-performance or stress conditions. Visual distribution analysis through Gaussian and CDF plots reinforced these quantitative findings, with HWiNFO maintaining more stable and centralized curves. While OHM occasionally showed improved sensitivity to spikes, it generally exhibited greater variability and deviated more from the baseline, particularly under heavy loads. These differences were especially critical in scenarios requiring high-frequency sampling or short-term fluctuation detection. Additionally, a significant discrepancy was observed between the results obtained on AMD and Intel CPU's. In all CPU stressor experiments on Intel systems, Intel Power Gadget produced broad, well-defined curves, while the other two meters struggled to replicate this pattern. In contrast, on AMD systems, both HWiNFO and OHM closely aligned with AMD uProf, suggesting that these meters may perform more reliably with AMD CPU's. Furthermore, both meters demonstrated strong performance during GPU stressor experiments, though they exhibited slight under-performance under high GPU load conditions.

Usability testing with 20 participants(split evenly between technical and non-technical backgrounds), highlighted HWiNFO's clear advantage in user experience. Through a TAM-based Likert-scale survey and task-based evaluation, HWiNFO scored higher across all categories which included, intention to use, perceived ease of use, usefulness, and satisfaction. Most users were able to complete all assigned tasks with HWInfo, whereas every OHM user failed at least one, typically due to missing features (e.g., logging or audio device monitoring). OHM's outdated UI, limited documentation, and difficult setup further contributed to lower scores. These results imply that while both tools can offer functional monitoring, HWiNFO stands out as the

more accurate, feature complete, and user-friendly option, thus better suited for consumers and professionals seeking comprehensive and reliable energy insights.

5.2 Reflection

Overall, this project provided a rich opportunity to explore technologies relevant to both hardware energy measurement and software-based monitoring tools. It allowed me to strengthen my skills as a data analyst, particularly in applying statistical techniques such as RMSE, CDF analysis, and Gaussian distribution fitting to compare the accuracy and consistency of different meters. I also developed a clearer understanding of how various meters operate under different workloads, and how their usability can vary dramatically depending on design and feature availability. Additionally, incorporating a vision-based LLM for data extraction accelerated the analysis process, especially when interpreting visual outputs like plots and screenshots, which proved highly effective for saving time.

That said, not all aspects of the project went smoothly. One of the biggest challenges was sourcing suitable smart meters as many options lacked proper data logging capabilities or only displayed real-time values without storage, which limited their utility in experiments. Similarly, traditional meters posed difficulties in data collection due to the absence of built-in logging features, requiring manual effort and increasing the risk of inconsistency. Additionally, the lack of comparison methodologies between smart meter and traditional meter had caused me to design my own tests which was difficult and time consuming. In the software meter evaluation, some unexpected results in the Intel-based experiments raised questions that couldn't be fully resolved due to time constraints, and a deeper investigation into those anomalies would have strengthened the analysis.

If I were to undertake this project again, I would focus on improving the experimental design in a few key ways. First, I would expand the scope by including more software meters, particularly tools available on Linux platforms, to diversify the comparison. Second, I would conduct more rigorous pre-testing of smart meters to ensure they offer the necessary data granularity and logging capabilities before committing to full-scale testing. Lastly, I would allocate more time to investigate outliers and edge cases in the data, especially in areas where software meter readings deviated from trusted baselines. These adjustments would not only enhance the robustness of the findings but also contribute to a more comprehensive evaluation of energy monitoring technologies.

5.3 Future work

While this project achieved its objectives in comparing the accuracy, consistency, and usability of various energy meters, several areas offer promising avenues for future exploration. Firstly, extending the evaluation to include a broader range of software meters(especially those available on alternative operating systems like Linux) would provide a more comprehensive comparison across platforms. Some meters may behave differently depending on system architecture and OS-level resource access, so cross-platform testing would offer deeper insights.

Secondly, a more granular investigation into the discrepancies observed in Intel-based software readings could uncover architectural or sensor-level factors that affect power reporting. This could be achieved by integrating more advanced profiling tools or accessing hardware counters directly. Additionally, performing long-duration tests under mixed workloads (e.g., background processes, intermittent GPU loads) could better simulate real-world scenarios and uncover trends not visible in short-term experiments.

Future work could also explore more consumer-grade smart meters with IoT integration and real-

time logging capabilities, comparing how cloud-connected meters stack up against traditional and software-based tools. This would test meters based on real world scenarios, thus uncovering much more valuable results for businesses or even consumers.

A Graphs related to traditional vs smart meters

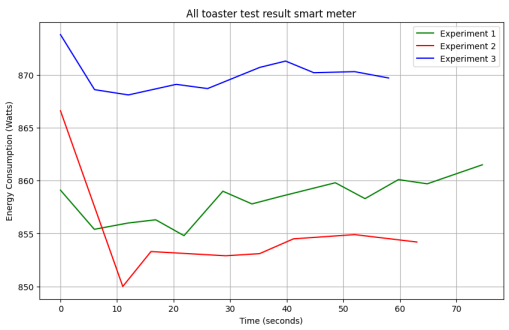


Figure A.1: *Smart Meter toaster experiment time series*

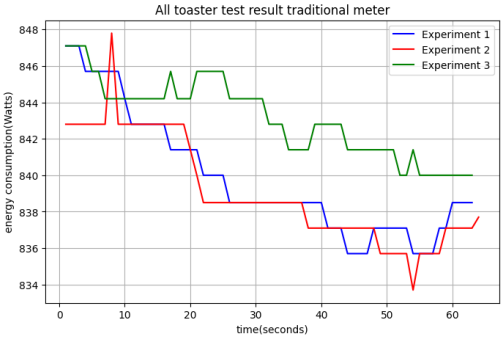


Figure A.2: *Traditional Meter toaster experiment time series*

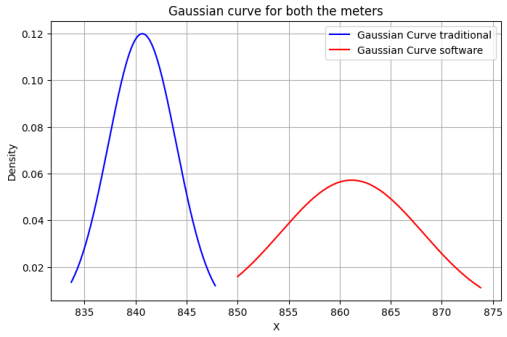


Figure A.3: *Gaussian Distribution of both meters*

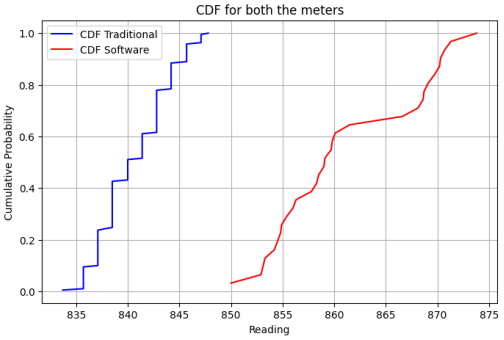


Figure A.4: *CDF of both meters*

Figure A.5: *Comparison of Smart and Traditional Meter Energy Measurements Across Different Analyses for toaster tests*

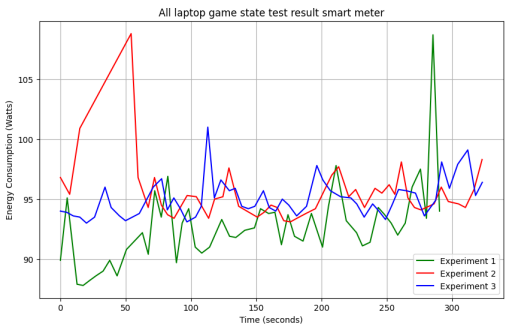


Figure A.6: *Smart Meter laptop game state experiment time series*

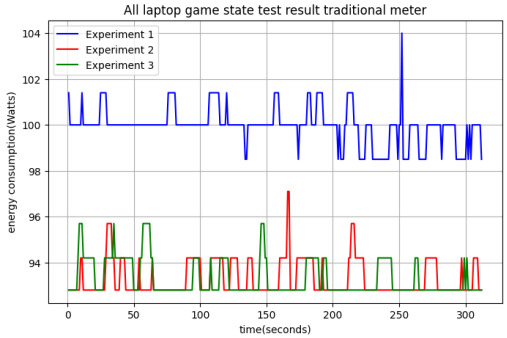


Figure A.7: *Traditional Meter laptop game state experiment time series*

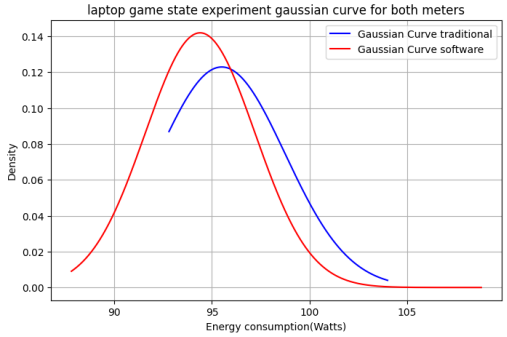


Figure A.8: *Gaussian Distribution of both meters*

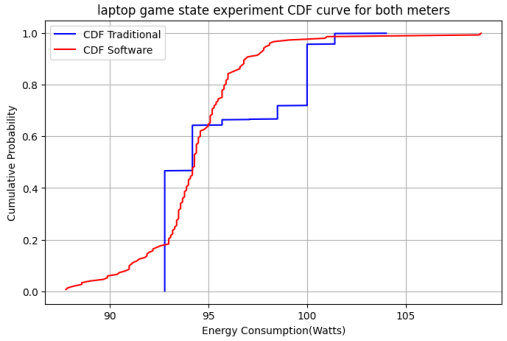


Figure A.9: *CDF of both meters*

Figure A.10: *Comparison of Smart and Traditional Meter Energy Measurements Across Different Analyses for laptop game state tests*

Tests	Smart meter observations	Traditional meter observations	Traditional meter vs smart meter comparison
Smart watch testing(low load)	Consistently recorded between 1.4W to 1.8W across all experiments; able to detect low load usage with minor fluctuations	Failed to detect readings in most cases; only registered one spike around 0.8W.	Smart meter captured low-power usage reliably and consistently. Traditional meter missed most readings due to low sensitivity.
Laptop idle state testing(low load)	Captured subtle fluctuations, reflecting background system activity.	Displayed flat, step-like pattern with limited variation.	Smart meter captured real-time idle variability; traditional meter under-represented due to limited sensitivity.
Rice cooker testing(medium load)	Consistent and smooth readings across experiments with tight fluctuation range.	Step-like readings with less granularity and slightly lower values overall.	Smart meter provided finer accuracy. Traditional meter lacked resolution, leading to under-representation. Both meter exceeded the rice cooker range of 250W.
Laptop running a game state testing(medium load)	Recorded fluctuations between 85W and 110W; more responsive to real-time variation with a broader Gaussian and smooth CDF; Experiment 2 showed notable spikes.	Captured values within same range but with step-like progression and limited variation; tighter Gaussian and step-like CDF reflect lower sensitivity.	Both meters aligned on average range, but smart meter better captured transient spikes and real-time changes, offering higher temporal resolution.
Laptop maximum CPU stressor testing(medium load)	Captured real-time fluctuations clearly; consistent range between 140W–160W.	Showed step-like pattern with occasional outliers below 100W and above 160W.	Smart meter was more stable and responsive. Traditional meter showed inconsistencies and possible under-reporting.
Toaster testing(high load)	Readings ranged from 850W to 870W; consistent across Experiments 1 and 2, but Experiment 3 showed isolated, higher values; broad Gaussian and wide CDF indicate variability.	Values remained tightly clustered within the expected 700W–850W range; showed step-like time series and limited sensitivity; consistent across all experiments.	Smart meter was less accurate, exceeding the rated power range of about 700W to 850W; traditional meter offered better precision for this test, though lacked high-resolution sampling.
Kettle testing(extremely high load)	Consistent readings across trials (2920W–2980W) with minor spikes. Narrow distribution reflects high measurement stability.	Fluctuating readings (2760W–2890W); broad distribution and signs of slight sampling issues due to step like curve in CDF.	Smart meter demonstrated greater consistency and precision, while the traditional meter showed higher variability and sensitivity issues under high load. Both remained within the kettle's rated range of 2520W–3000W.

Table A.1: *summary of traditional meter vs smart meter tests in table*

B Graphs related to software vs software meters in Intel CPU

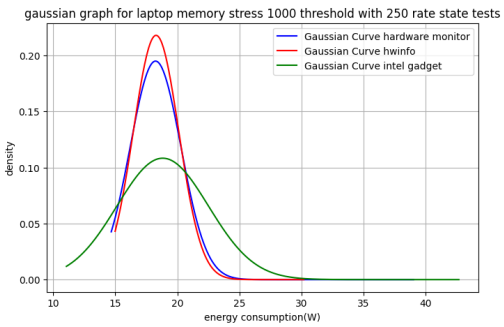


Figure B.1: *Software meters gaussian curve*

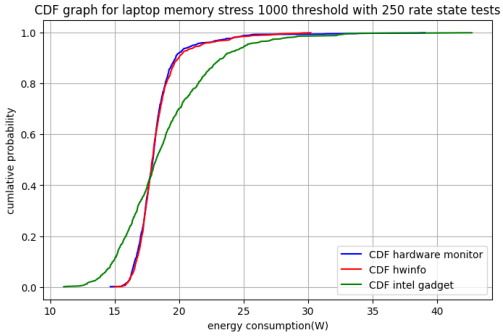


Figure B.2: *Software meters CDF curve*

Figure B.3: *Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU for memory stressor test of 1000 threshold and 250 rate*

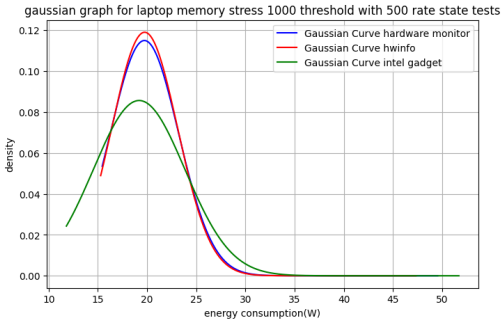


Figure B.4: *Software meters gaussian curve*

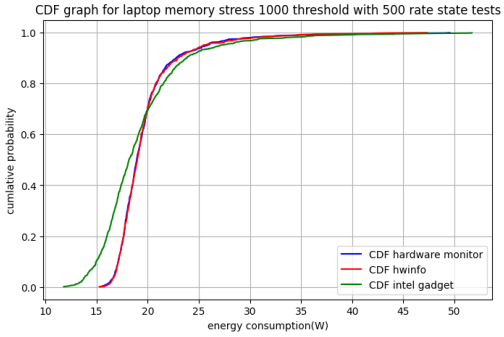


Figure B.5: *Software meters CDF curve*

Figure B.6: *Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU for memory stressor test of 1000 threshold and 500 rate*

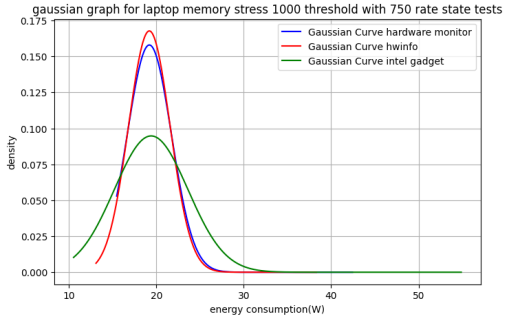


Figure B.7: *Software meters gaussian curve*

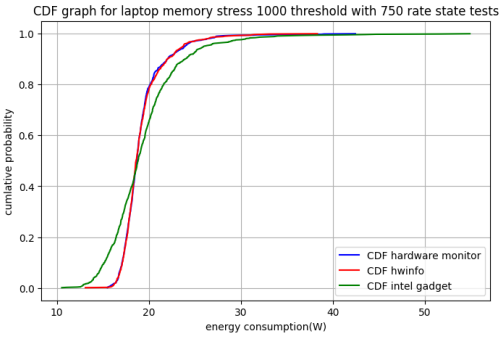


Figure B.8: *Software meters CDF curve*

Figure B.9: *Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU for memory stressor test of 1000 threshold and 750 rate*

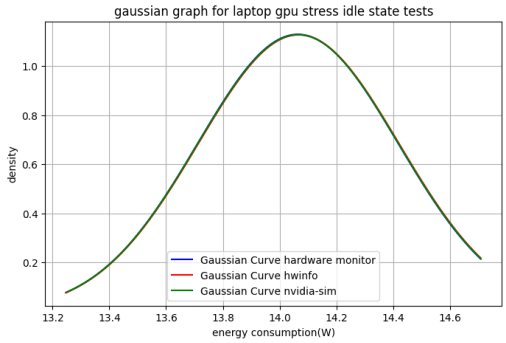


Figure B.10: *Software meters gaussian curve*

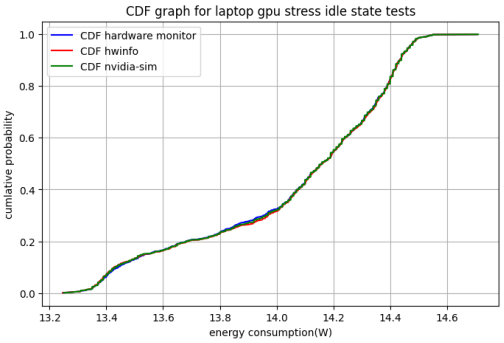


Figure B.11: *Software meters CDF curve*

Figure B.12: *Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU for GPU stressor test of idle state*

C | Graphs related to software vs software meters in AMD CPU

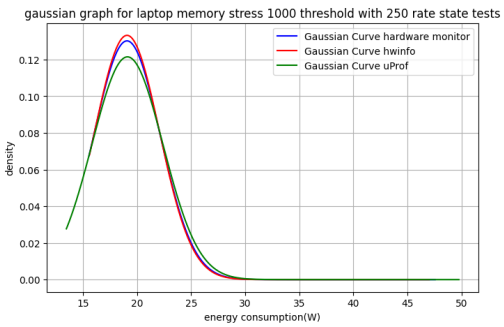


Figure C.1: Software meters gaussian curve

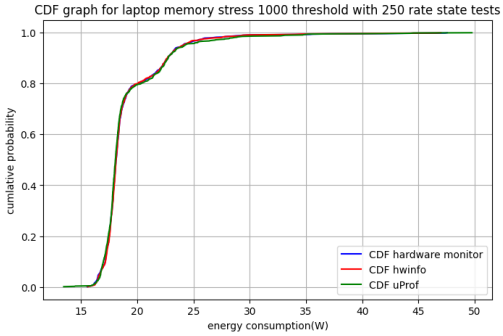


Figure C.2: Software meters CDF curve

Figure C.3: Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU for memory stressor test of 1000 threshold and 250 rate

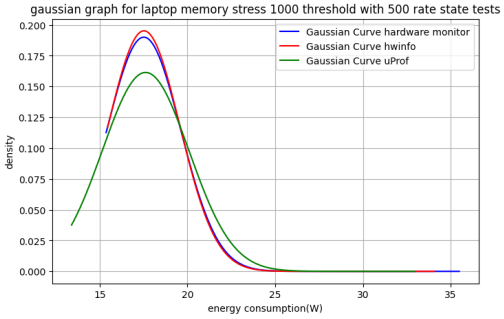


Figure C.4: Software meters gaussian curve

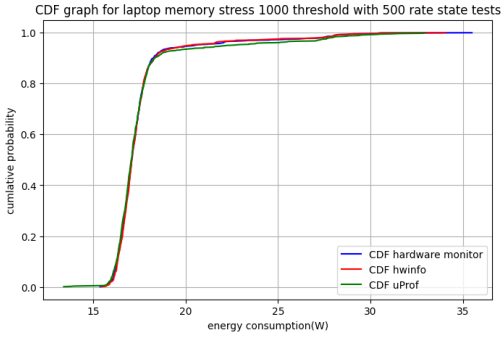


Figure C.5: Software meters CDF curve

Figure C.6: Comparison of Software Meters Energy Measurements Across Different Analyses in Intel CPU for memory stressor test of 1000 threshold and 500 rate

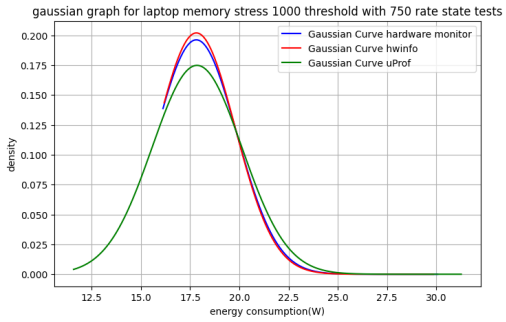


Figure C.7: *Software meters gaussian curve*

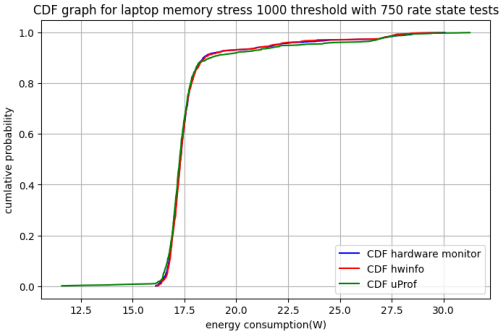


Figure C.8: *Software meters CDF curve*

Figure C.9: *Comparison of Software Meters Energy Measurements Across Different Analyses in AMD CPU for memory stressor test of 1000 threshold and 750 rate*

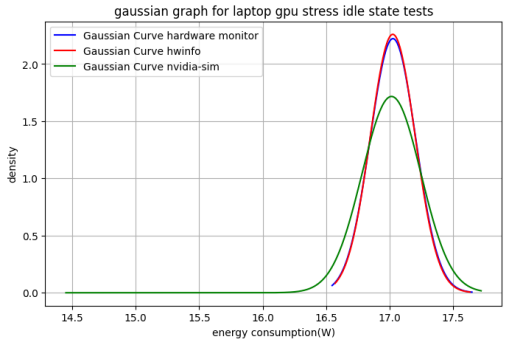


Figure C.10: *Software meters gaussian curve*

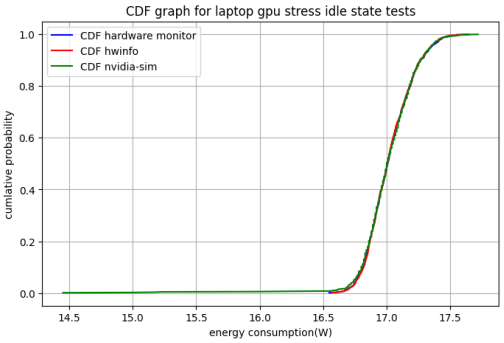


Figure C.11: *Software meters CDF curve*

Figure C.12: *Comparison of Software Meters Energy Measurements Across Different Analyses in AMD CPU for GPU stressor test of idle state*

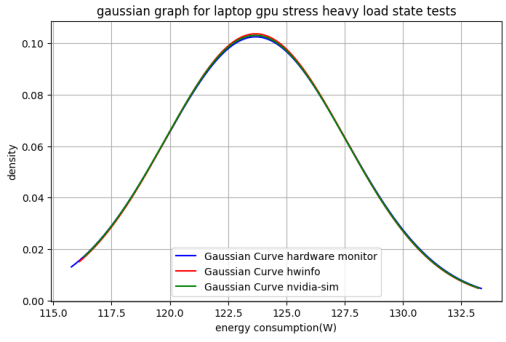


Figure C.13: *Software meters gaussian curve*

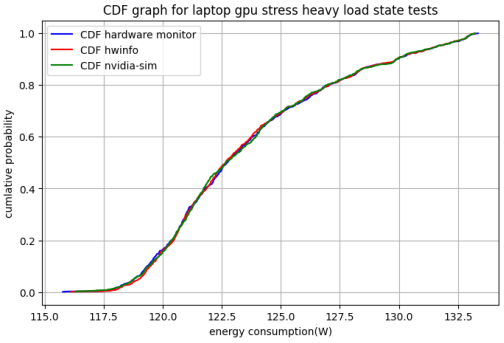
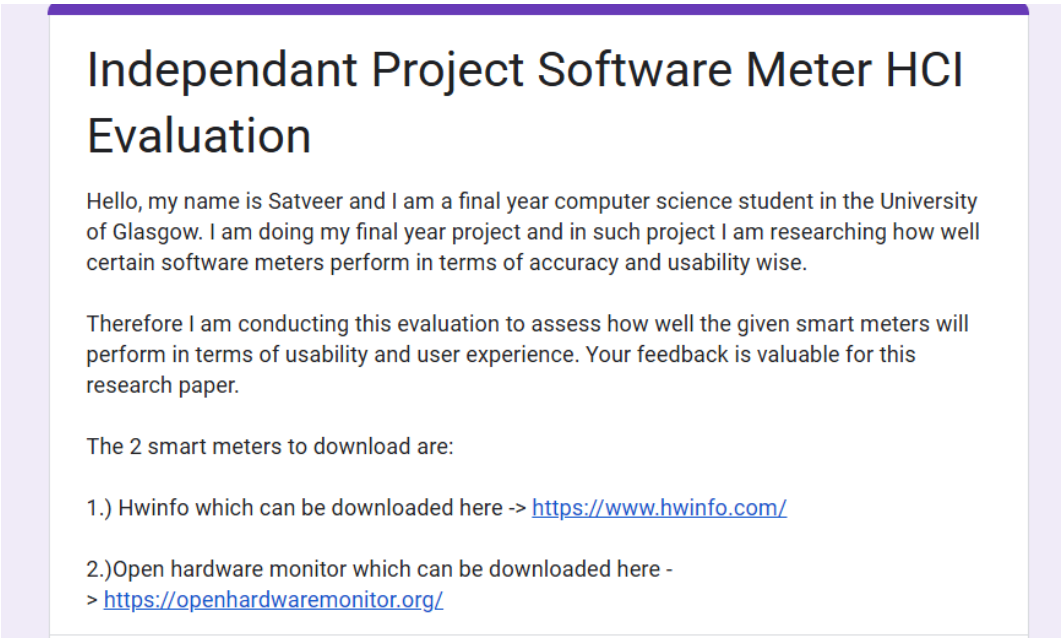


Figure C.14: *Software meters CDF curve*

Figure C.15: *Comparison of Software Meters Energy Measurements Across Different Analyses in AMD CPU for GPU stressor test using heavy load GPU stressor*

D Google Survey questions screenshot



Independant Project Software Meter HCI Evaluation

Hello, my name is Satveer and I am a final year computer science student in the University of Glasgow. I am doing my final year project and in such project I am researching how well certain software meters perform in terms of accuracy and usability wise.

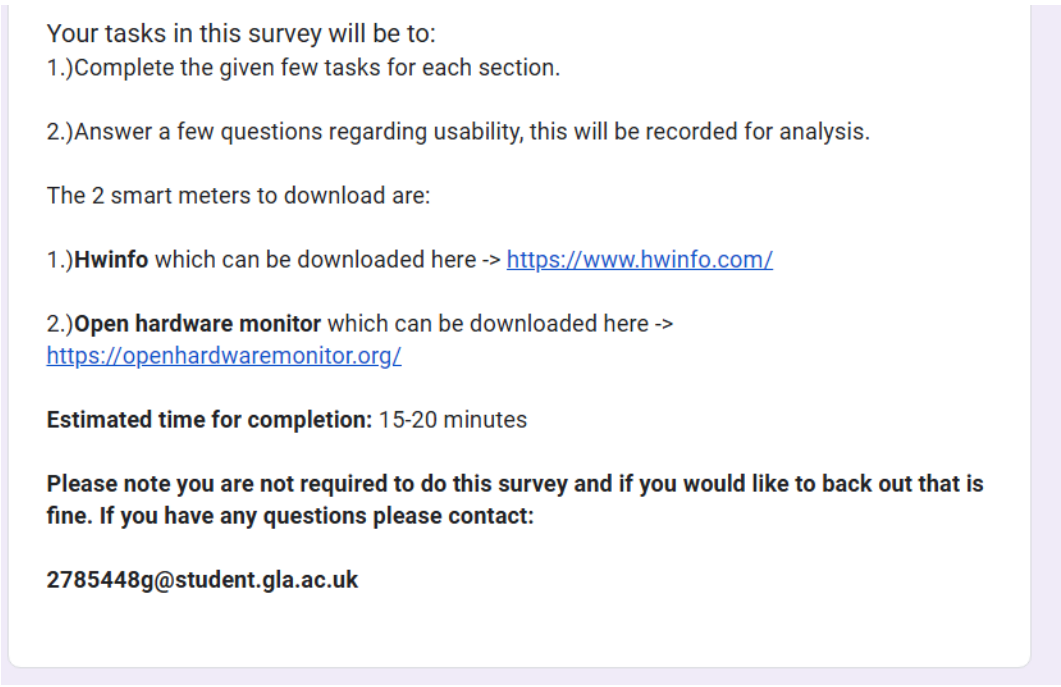
Therefore I am conducting this evaluation to assess how well the given smart meters will perform in terms of usability and user experience. Your feedback is valuable for this research paper.

The 2 smart meters to download are:

1.) Hwinfo which can be downloaded here -> https://www.hwinfo.com/

2.)Open hardware monitor which can be downloaded here -
> https://openhardwaremonitor.org/

Figure D.1: Usability survey introduction



Your tasks in this survey will be to:
1.)Complete the given few tasks for each section.

2.)Answer a few questions regarding usability, this will be recorded for analysis.

The 2 smart meters to download are:

1.)**Hwinfo** which can be downloaded here -> https://www.hwinfo.com/

2.)**Open hardware monitor** which can be downloaded here ->
https://openhardwaremonitor.org/

Estimated time for completion: 15-20 minutes

Please note you are not required to do this survey and if you would like to back out that is fine. If you have any questions please contact:

2785448g@student.gla.ac.uk

Figure D.2: Usability survey task introduction

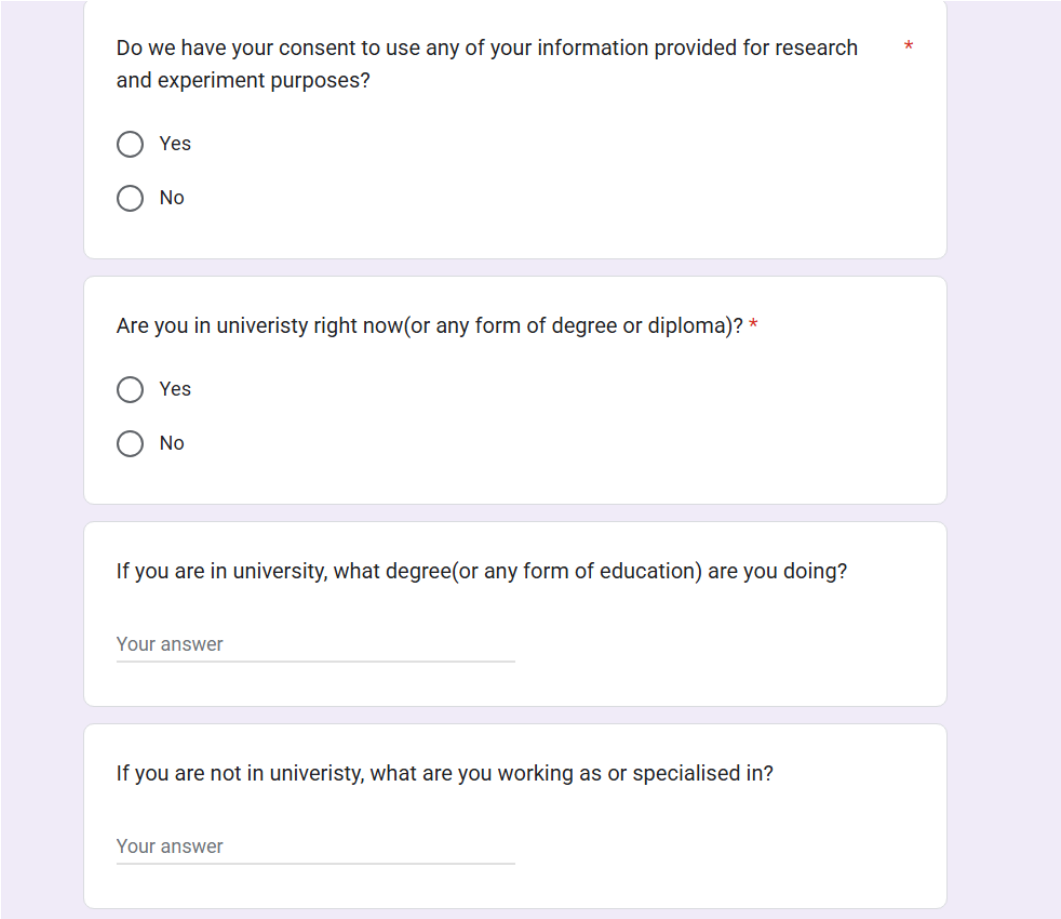


Figure D.3: *Usability survey tasks introductory questions*

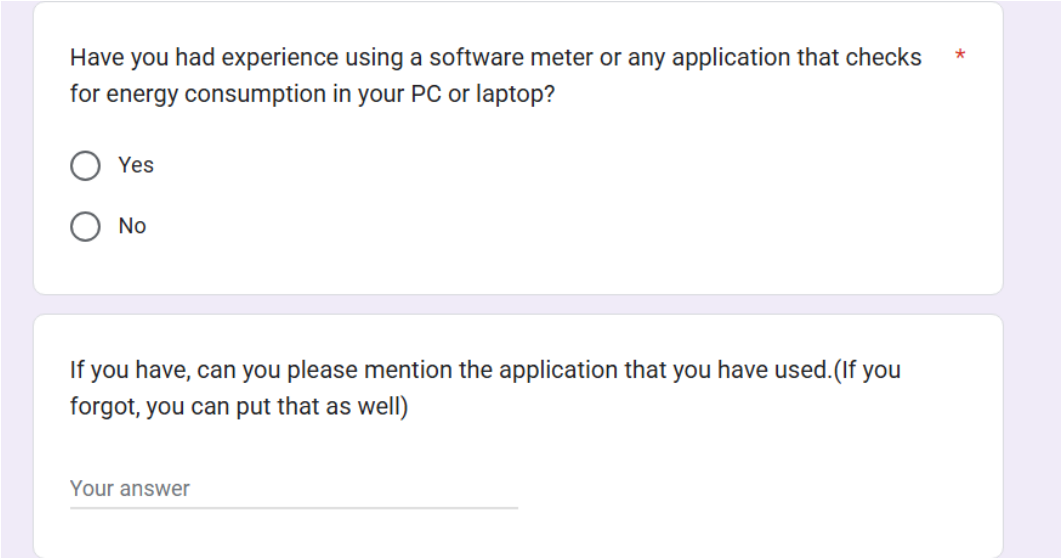


Figure D.4: *Usability survey tasks introductory questions part 2*

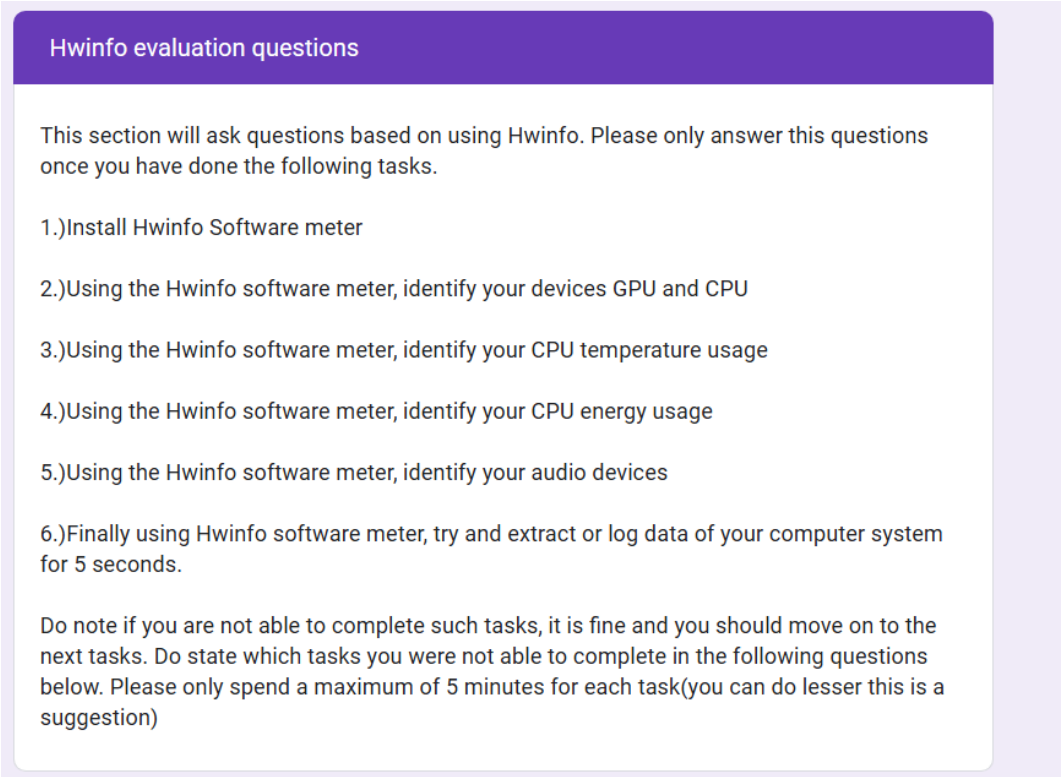


Figure D.5: *Usability survey HWiNFO tasks*

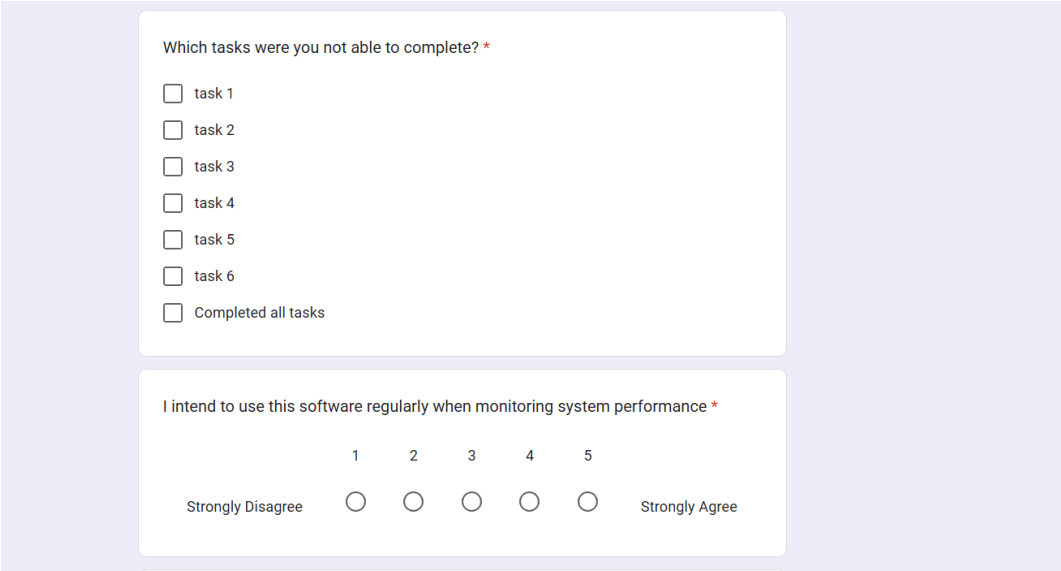


Figure D.6: *Usability survey HWiNFO questions*

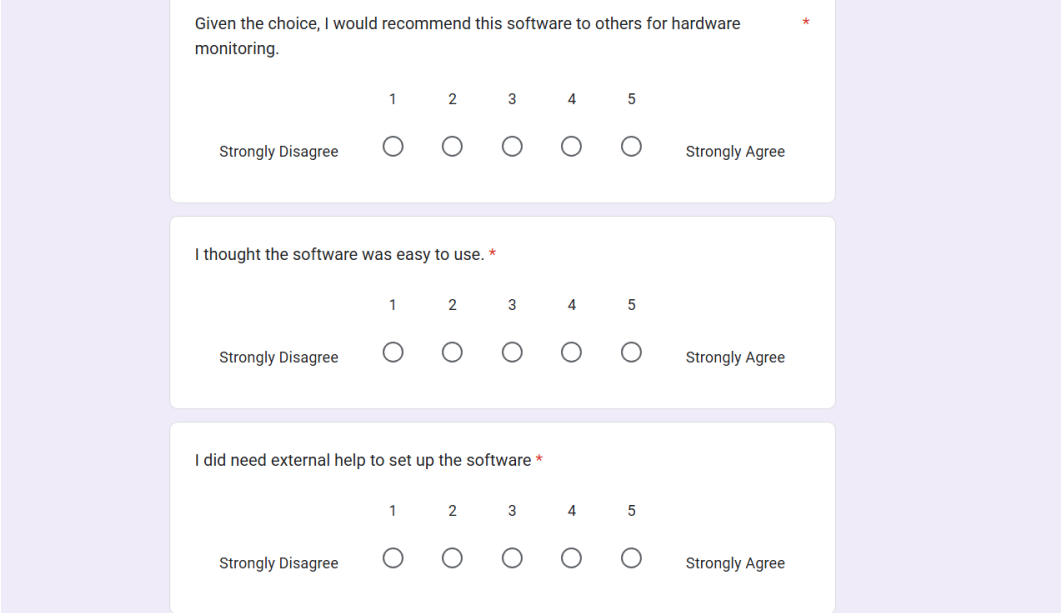


Figure D.7: *Usability survey HWiNFO questions part 2*

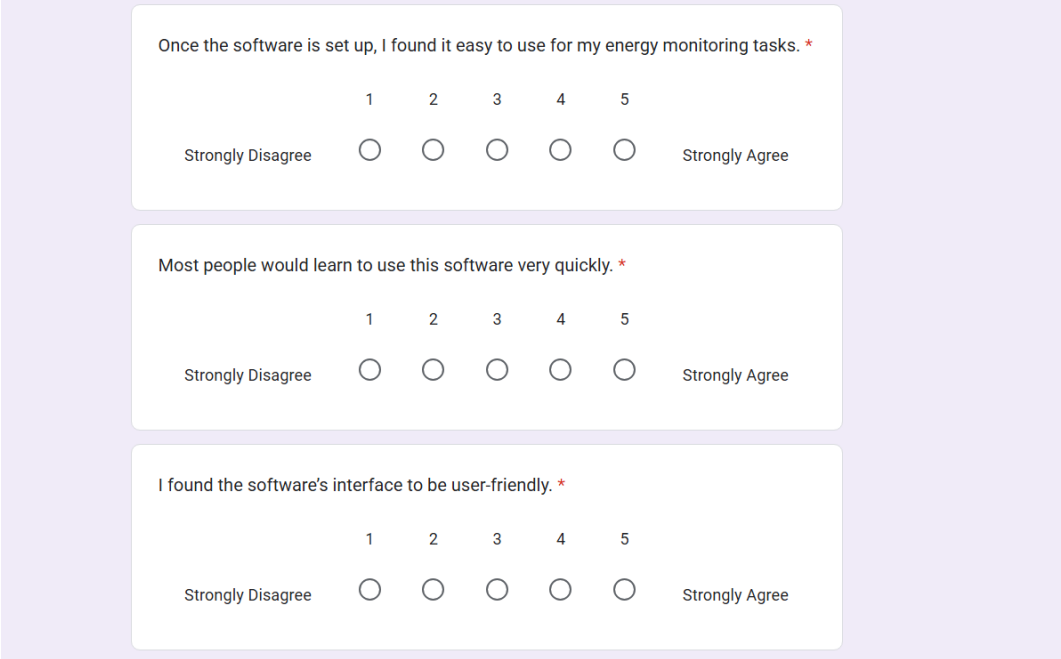


Figure D.8: *Usability survey HWiNFO questions part 3*

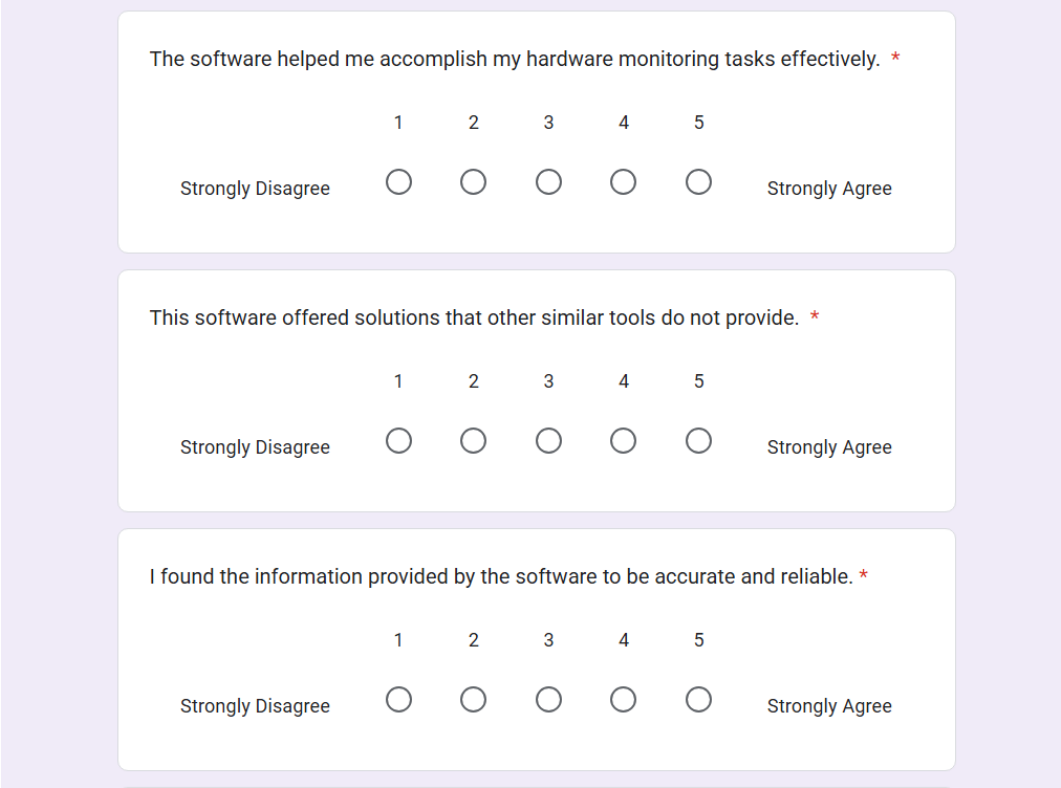


Figure D.9: *Usability survey HWiNFO questions part 4*

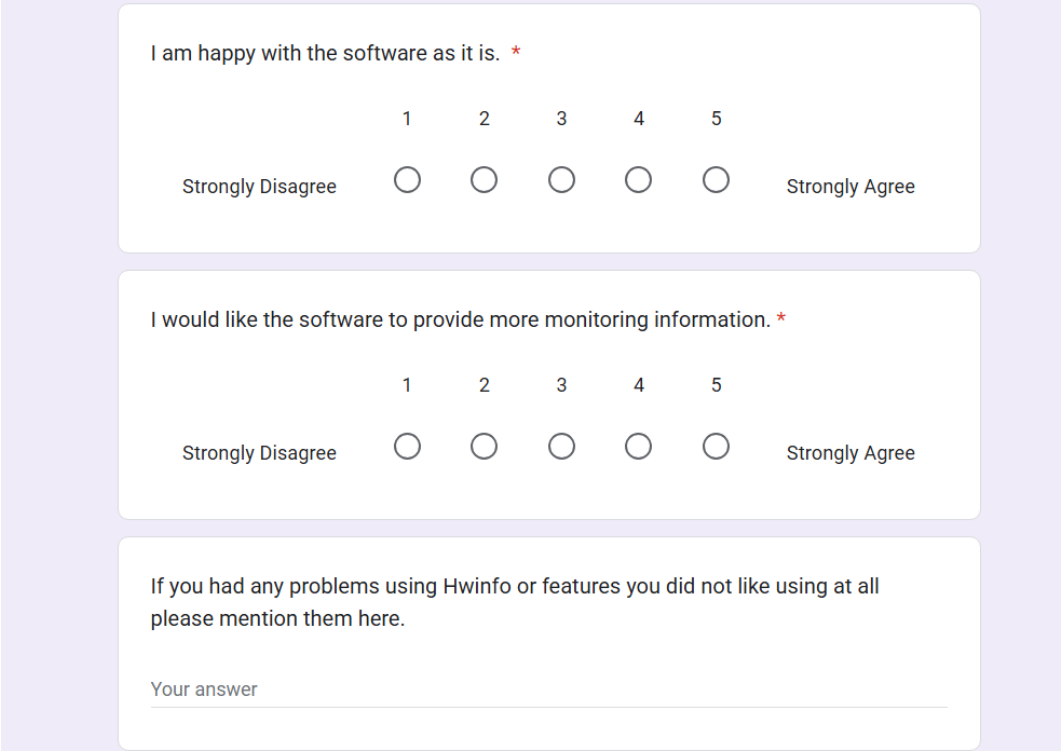


Figure D.10: *Usability survey HWiNFO questions part 5*

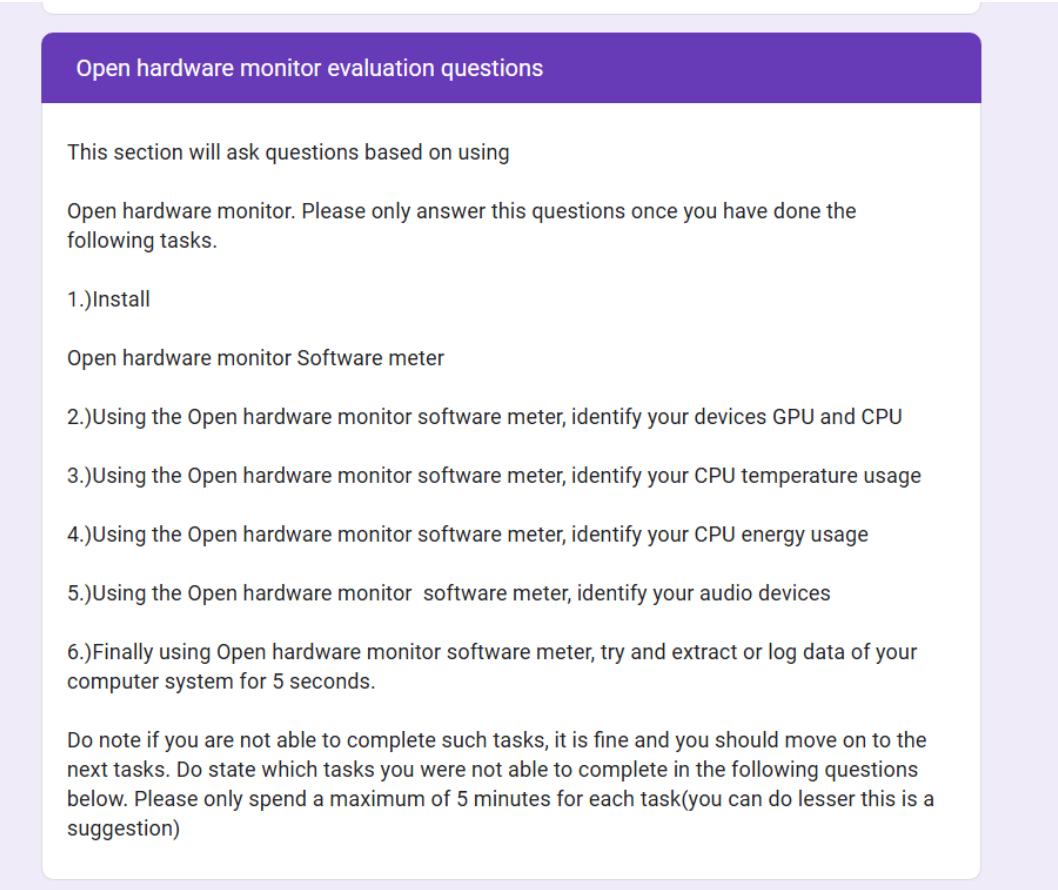


Figure D.11: *Usability survey Open Hardware Monitor tasks*

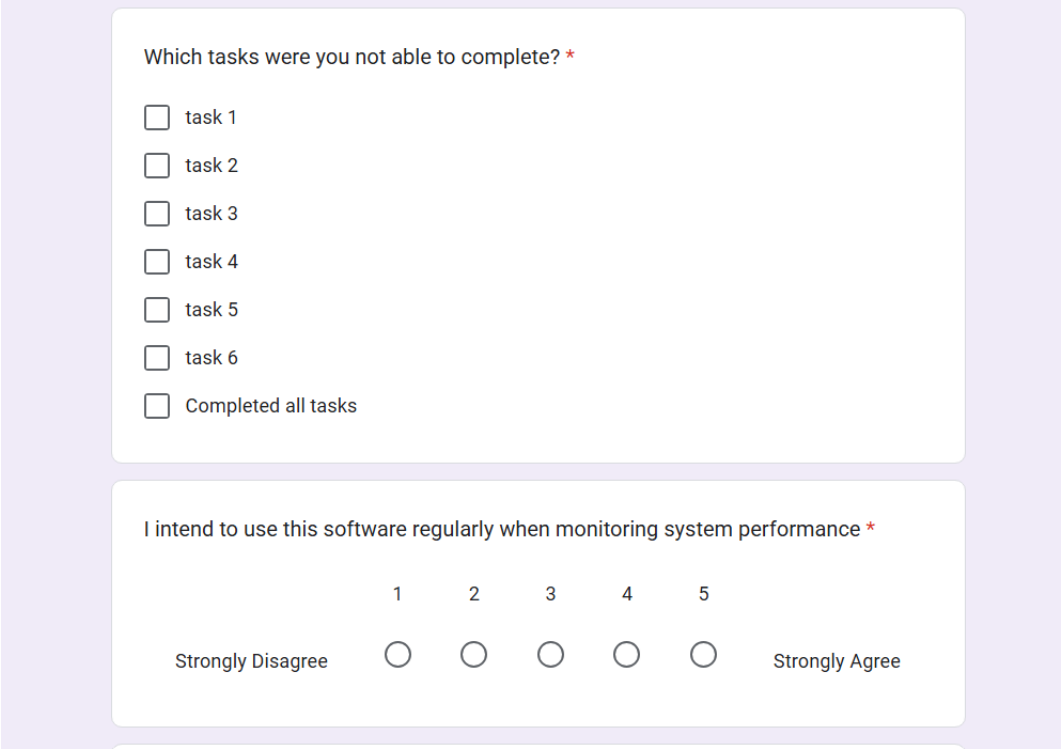


Figure D.12: *Usability survey Open Hardware Monitor questions part 1*

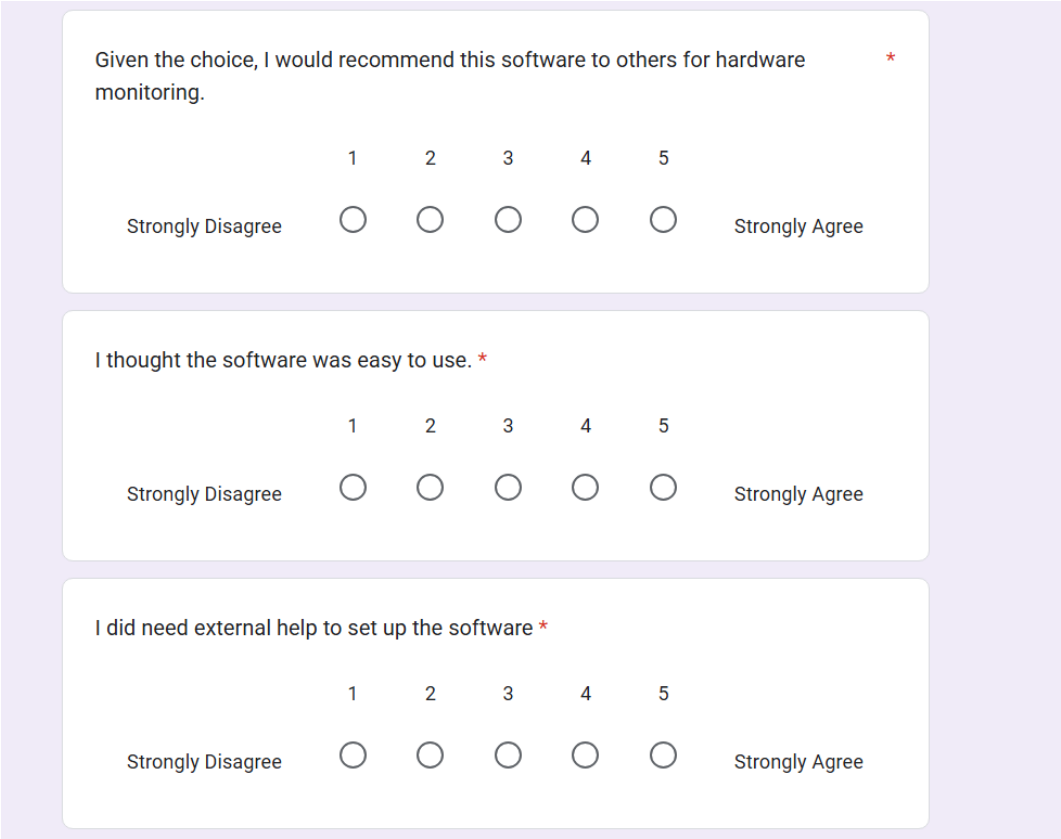


Figure D.13: *Usability survey Open Hardware Monitor questions part 2*

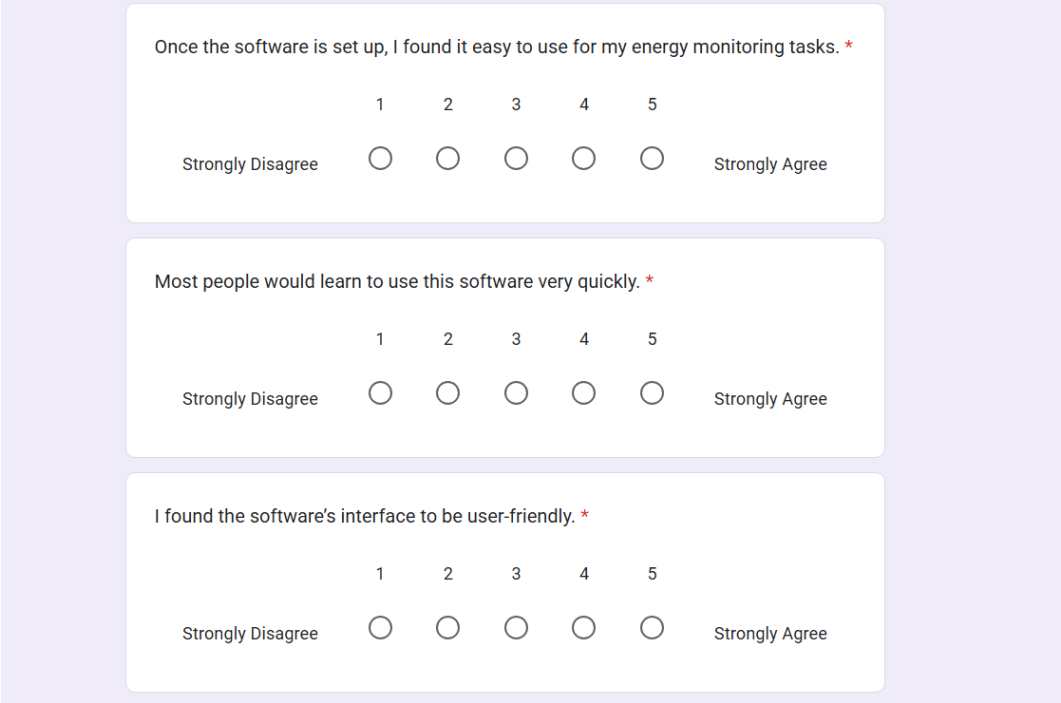


Figure D.14: *Usability survey Open Hardware Monitor questions part 3*

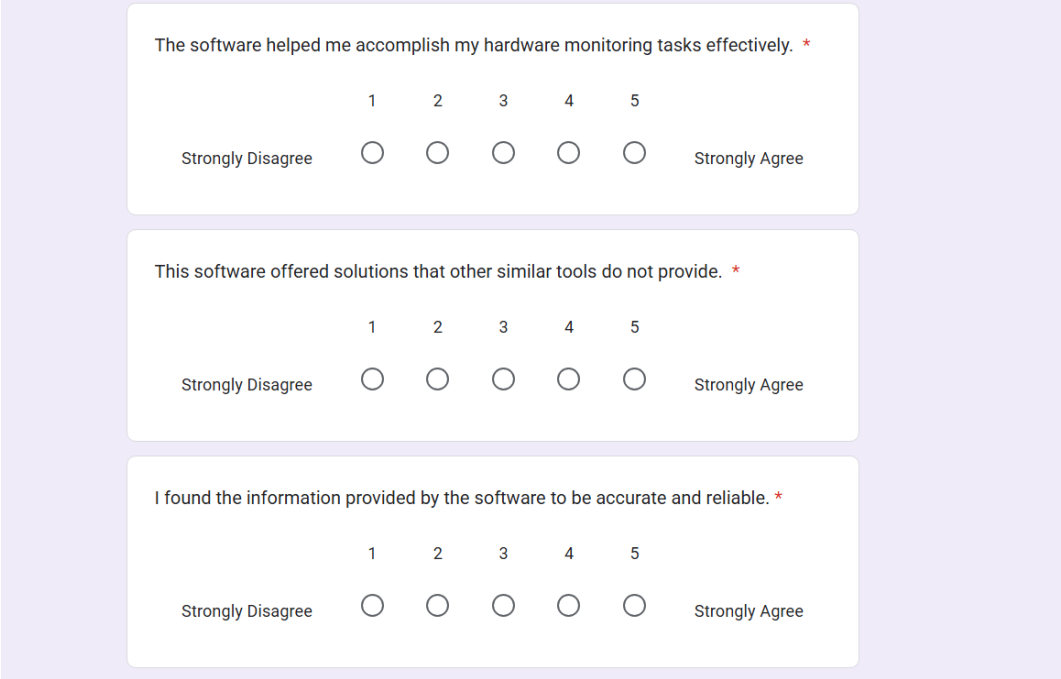


Figure D.15: *Usability survey Open Hardware Monitor questions part 4*

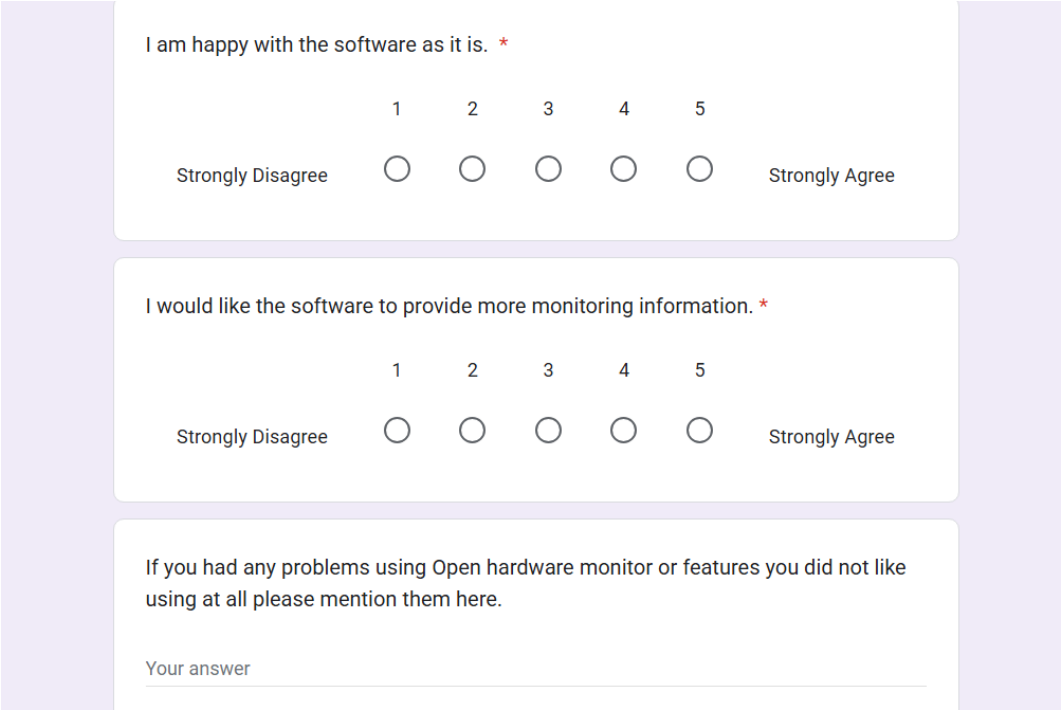


Figure D.16: *Usability survey Open Hardware Monitor questions part 5*

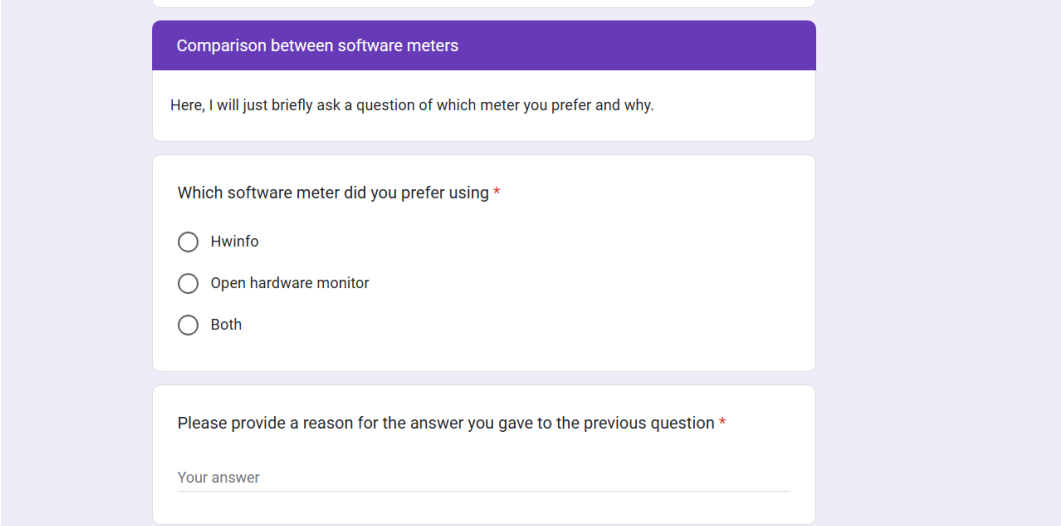


Figure D.17: *Usability survey comparison questions*

E User manual to run the code

This project has multiple folders, I will go through the main folders which basically include the code and data.

There are 4 main files:

1.) `experiment_data_amd/` → stores all data and code regarding experiments done in AMD CPU

2.) `experiment_data_intel/` → stores all data and code regarding experiments done in Intel CPU

3.) `smart_and_trad_results/` → stores all data and code regarding experiments done with traditional meter vs smart meter

4.) `usability_testing_results/` → stores all data and code regarding usability experiments

These 4 files have their own code in which I will go through and the raw experiment data sorted according to the file names. One main thing to note is most of the instructions to run the code is in the notebook, therefore the manual guide in most of these folders will refer to such notebook. The only main required thing you will need to do is download the required packages as mentioned per the read me.

Bibliography

Baah, B. and Nwagbo, C. (2022), 'A comparative study of basic types of energy meters and metering', *Global Journal of Engineering and Technology Advances* **10**, 117–124.

Bimenyimana, S. and Asemota, G. (2018), 'Traditional vs smart electricity metering systems: A brief overview', *Journal of Marketing and Consumer Research* **46**. An International Peer-reviewed Journal. ISSN 2422-8451. Last accessed: 2025-03-27.

Durand, A., Hirzel, S., Rohde, C., Gebele, M., Lopes, C., Olsson, E. and Barkhausen, R. (2022), 'Electric kettles: An assessment of energy-saving potentials for policy making in the european union', *Sustainability* **14**(20). Last accessed: 2025-03-08.

Fieni, G., Acero, D., Rust, P. and Rouvoy, R. (2024), 'Powerapi: A python framework for building software-defined power meters', *Journal of Open Source Software* **9**, 6670.

Fox, J. E. (2015), The science of usability testing, *in* 'Proceedings of the 2015 Federal Committee on Statistical Methodology (FCSM) Research Conference'. Last accessed: 2025-03-27.

Good Energy (2025), 'What to do if you think your meter is faulty', `https://www.goodenergy.co.uk/support/articles/what-to-do-if-you-think-your-meter-is-faulty/`. Last accessed: 2025-03-08.

International Energy Agency (IEA) (2025), 'Electricity 2025', `https://www.iea.org/reports/electricity-2025`. Licence: CC BY 4.0. Last accessed: 2025-03-27.

IoT For All (2025), 'Smart meter accuracy: How reliable are they?', `https://www.iotforall.com/smart-meter-accuracy`. Last accessed: 2025-03-01.

Jay, M., Ostapenco, V., Lefevre, L., Trystram, D., Orgerie, A.-C. and Fichel, B. (2023), An experimental comparison of software-based power meters: focus on cpu and gpu, *in* '2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)', pp. 106–118.

Khan, K. N., Hirki, M., Niemi, T., Nurminen, J. K. and Ou, Z. (2018), 'Rapl in action: Experiences in using rapl for power measurements', *ACM Transactions on Modeling and Performance Evaluation of Computing Systems* **3**(2). Last accessed: 2025-03-27.

Khattak, A., Khanji, S. and Khan, W. (2019), Smart meter security: Vulnerabilities, threat impacts, and countermeasures, *in* 'Proceedings of the 13th International Conference on Ubiquitous Information Management and Communication (IMCOM) 2019', Springer International Publishing, Cham, pp. 554–562.

LinkedIn (2025), 'How can you accurately test smart meters?', `https://www.linkedin.com/advice/0/how-can-you-accurately-test-smart-meters-youye#:~:text=Comparison%20testing%20is%20one%20such,%2C%20power%20quality%2C%20and%20harmonics`. Last accessed: 2025-03-08.

Nielsen, J. (2025), 'Ux without user research is not ux', https://www.nngroup.com/articles/ux-evidence/. Last accessed: 2025-03-08.

Olivares Rojas, J. C., Reyes-Archundia, E., Gutierrez Gnecchi, J. A., Molina-Moreno, I., Ramos, J. and Serna, G. (2020), 'Towards human-computer interaction on smart metering systems', *Avances en Interacción Humano-Computadora* **1**, 90–93.

Poudineh, R. (2025), 'Global electricity demand: what's driving growth and why it matters? - oxford institute for energy studies', https://www.oxfordenergy.org/publications/global-electricity-demand-whats-driving-growth-and-why-it-matters/. OIES Energy Comment. Last accessed: 2025-03-27.

Power Line (2021), 'Smart meter testing', https://powerline.net.in/2021/03/07/smart-meter-testing-2/. Last accessed: 2025-03-08.

PricewaterhouseCoopers (PwC) (2025), 'Cloud and ai business survey', https://www.pwc.com/us/en/tech-effect/cloud/cloud-ai-business-survey.html. Last accessed: 2025-03-01.

Raza, M., Rind, Y., Javed, I., Zubair, M., Mehmood, M. Q. and Massoud, Y. (2023), 'Smart meters for smart energy: A review of business intelligence applications', *IEEE Access* **PP**, 1–1.

Sharman, C. (2025), 'Smart meters vs. traditional energy meters: Key differences and benefits', https://www.saga.co.uk/magazine/homes/smart-versus-traditional-energy-meters. Last accessed: 2025-03-01.

UK Government (2023), 'Q4 2023 smart meters statistics report', https://assets.publishing.service.gov.uk/media/65fc3d0a65ca2f001b7da7c5/Q4_2023_Smart_Meters_Statistics_Report.pdf. Last accessed: 2025-03-01.

Wang, T., Zhao, Q., Gao, W. and He, X. (2024), 'Research on energy consumption in household sector: a comprehensive review based on bibliometric analysis', *Frontiers in Energy Research* **11**.